# Confidential and verifiable upstream inference over standard TLS 1.3

A no-trusted-hardware protocol for LLM routing nodes

Juraj Bednar <juraj@bednar.io>

Coauthors: Claude Opus 5.5, Claude Sonnet 5.5, Kimi K3,
DeepSeek V4.1 Flash, GPT-6 Astra, GLM-5.3 Flash

2026-11-11

**Abstract**

LLM routing nodes forward user requests to upstream providers with the node's own API key, so users must trust the node with their prompts, and the node must keep its key from its users. Enclave-based designs ask every user to verify an attestation of the node's software after every update. Our protocol rests instead on the provider's TLS certificate and on cryptography. The client is the TLS 1.3 client of the provider and the node relays the records. Client and node run the TLS key schedule as a two-party computation (share conversion over P-256, then a garbled circuit that only the node garbles and the client checks gate by gate with information-theoretic MACs): the node alone gets the key of the first application epoch, in which it writes its credential and a KeyUpdate, and the client alone gets the keys for the requests and the streamed responses. Nobody holds the ECDHE or master secret, so the client cannot recover the API key even if it records the node's traffic. In the previous version of the protocol every request paid for such a handshake. Here the unit is a connection. On HTTP/2 the node's epoch contains a probe request whose `authorization` header enters the provider's header-compression table, and every later request, written by the client, names the credential by a one-byte index without learning its value. The node releases a client record to the provider only after a zero-knowledge proof that the record is the HTTP/2 frame it must be and that the request ends in a public suffix pinning the model and token cap; a second proof shows that the billed usage is the provider's own report for that request. Where HTTP/2 is not available the same lifecycle carries one HTTP/1.1 request per connection. A split of HMAC at the compression function shrinks the garbled key schedule from 31 to 22 SHA-256 compressions, at the cost of making inner hash states public. A handshake moves 16.4 MB from the node to the client and 1.1 MB back, about 7 s on a home link with a 2 Mbit/s uplink. Against a live provider, a later request on an open connection moved 0.5 to 1.0 MB and was released to the provider about 60 ms after it was sent. The new parts (the frame rules, the window proof, the split key schedule) have had no external review.

## Contents

**1. Introduction** **2**
1.1 Contributions . . . 3

**2. Model** **4**

**3. Preliminaries and the provider profile** **4**
3.1 Provider profile . . . 4
3.2 TLS 1.3 key schedule . . . 7
3.3 Cryptographic building blocks . . . 7

**4. Protocol** **8**
4.1 Who holds which secret . . . 8
4.2 Connections, framings and epochs . . . 8
4.3 Setup, preprocessing and the joint key share . . . 10

4.4 The two-party key schedule . . . 10
4.5 Connection head (epoch 0) and $\pi_{NK}$ . . . 13
4.6 Requests (epoch 1) and $\pi_{WIN}$ . . . 14
4.7 Response and usage disclosure ($\pi_{C3S}$) . . . 15
4.8 Settlement . . . 17
4.9 Aborts . . . 18
4.10 Image generation . . . 19

**5. Security analysis** **19**

**6. Implementation and evaluation** **24**
6.1 Prototype . . . 24
6.2 Component costs . . . 25
6.3 End to end . . . 26
6.4 Over a network and bandwidth . . . 27
6.5 Live verification . . . 28
6.6 The dual-execution version . . . 28

**7. Earlier versions** **29**
7.1 From v2 to v3 . . . 29
7.2 v1: the single-party variant . . . 30

**8. Deployment pattern** **31**
8.1 Denial-of-service protection and fees . . . 31

**9. Related work and alternatives** **32**

**10. Conclusion** **34**

**Appendix A. Case study: the provider profile in practice** **34**

**References** **35**

# 1. Introduction

Routing nodes and gateway services sit between users and upstream LLM APIs. A node holds an upstream account and forwards requests with its own API key; users pay the node (typically prepaid with e-cash tokens such as Cashu), and the node pays the upstream. Two trust problems follow.

**The user trusts the node with everything.** The node sees the prompt and the response; it can log, alter or replay them. This is a privacy problem (prompts contain source code, contracts, medical text) and a correctness problem (the node can substitute models or parameters).

**The node must keep its credential from the user, and both must agree on the bill.** The node's upstream API key must never leak to users. Charging must be defensible: the user should be able to check that the tokens billed are the tokens the upstream actually produced and reported.

The usual answer to the first problem is confidential computing: run the node, or the model, inside a trusted execution environment (TEE) and let users verify a remote attestation. That moves trust from the operator to the hardware vendor and to the measured software. For the guarantee to mean anything, a user has to know which software measurement is acceptable, which requires a reproducible build and a review of the whole stack that touches the plaintext, and has to repeat that after every update. In practice few users do this, and the attestation becomes a badge rather than a check.

This paper takes a narrower route. A user's request to a provider is already protected by TLS, and the provider is already authenticated by a WebPKI certificate. We **hand the TLS client role to the user and let the node relay ciphertext**. The node contributes its credential to the request without the user learning it, and cryptographic proofs pin the few

bytes each side may write and bind billing to the provider's own numbers. There is no attested software to track: the user checks a certificate and a handful of proofs whose statements are fixed by the protocol.

The difficulty is the credential. If the user's client runs the TLS handshake alone, it knows every traffic secret, including the key of the record that carries the node's API key; the key is then hidden only as long as the client never sees that record's ciphertext. We remove that condition by computing the handshake key schedule jointly: the node obtains the key for its own record, the client obtains the keys for the requests and the responses, and neither can derive the other's. The two-party computation (2PC) covers only the key schedule, a fixed workload of 22 garbled SHA-256 compressions, never the payload.

A garbled key schedule is single-use, so the node can write its credential once per handshake. If every request needs the credential in its head, every request needs a handshake, and the handshake is the expensive part: tens of megabytes of garbled tables. The second idea of this paper is to make one write serve many requests. HTTP/2 compresses headers against a table that the receiver keeps for the connection. The node writes, in its own epoch, a request whose `authorization` header is stored in that table. From then on the client can write complete requests that refer to the stored header by its index, and the provider authenticates them, while the client still does not know the key. What the client must then prove is that its records are exactly the frames the node expects, since a frame is an instruction to the provider's HTTP/2 stack and the node cannot read it.

## 1.1 Contributions

1. A TLS 1.3 protocol (section 4) in which the first application epoch belongs to the relay node, which writes its credential and a KeyUpdate under a key only it holds, while the requests and the responses run under keys only the client holds. The handshake key schedule runs as a two-party computation secure against a malicious peer, in which only the node garbles and the client checks every gate against masks the node has authenticated to it. Neither party ever reveals its inputs; a cheating node can learn at most 3 bits per connection of additional selective-failure leakage about values used only in that connection, in an idealized abort model, provided the semi-honest share conversion at its start gives it nothing beyond its outcome and the garbling argument holds.
2. A split of the key schedule at the SHA-256 compression function (section 4.4): only the pad states of four secret keys and the outer compressions are garbled, the inner states are public and both parties compute the inner digests in the clear. This cuts the circuit from 656,179 to 479,022 AND gates and the client's download from 22.1 to 16.4 MB, and rests on the two compression-function assumptions of Bellare's HMAC proof, used directly, plus collision resistance of the compression function under a known chaining value for the distinct labels of one key (section 4.4); the composed argument has not been written as a proof.
3. A way to use a credential many times after writing it once: the node stores it in the provider's HPACK dynamic table in its own epoch, and the client's later HTTP/2 requests name it by index (sections 4.2, 4.5), with an argument that the stored value cannot be read back (section 5).
4. A request-integrity construction for HTTP/2 and HTTP/1.1: one statement, $\pi_{WIN}$, proves plaintext windows of the client's records equal to public bytes, and a set of frame rules makes those windows sufficient: one frame per record, the frame header and the absence of TLS padding proven for every record, the header block proven in full, and the body's tail proven to be an agreed public JSON suffix that pins every cost-relevant parameter (section 4.6). The provider sees no record before its proof verifies. The node proves that its hidden record matches a published template (section 4.5).
5. A verifiable-accounting construction: the client proves that the upstream's terminal usage event is authentic decrypted output of the connection, inside the request's own interval of server records, and the node charges exactly that usage at the rates of the offer the client verified, through its ordinary billing (sections 4.7, 4.8). Proofs that decrypt reveal one bit (equal or not), never decrypted bytes.
6. A **provider profile** (section 3.1) that separates what any upstream must satisfy from provider-specific details, including what the HTTP/2 framing needs.
7. A working implementation verified end to end against a live provider, with measurements of the components and of the whole path (section 6), and a comparison with the previous versions of the protocol (section 7).

# 2. Model

Parties:

- **C, client** (a user's daemon or app). TLS 1.3 client of the upstream connection; the only holder of the request and response traffic secrets; payer of the node.
- **N, node.** Holds the upstream API key; owns the TCP connection to the upstream and relays it; the only holder of the first application epoch's key; bills C through its ordinary accounts.
- **U, upstream** (any provider whose profile, section 3.1, is satisfied). Unmodified.

Channels: C to N is a control channel, a byte relay and one channel per proof or 2PC (in our deployment, WebSockets); N to U is ordinary TCP:443.

Goals:

| ID | Property |
|---|---|
| P1 | Credential confidentiality: only N knows the upstream API key. C learns nothing about it even if it sees every byte N sends to U. |
| P2 | Content confidentiality: only C (and U) can read the prompt and the response. N sees ciphertext plus the public billing data: the usage event, or on an upstream error the response head. |
| P3 | Integrity: C can verify that the request U received is C's own content behind the agreed public template content (not its exact wire bytes: N's epoch-0 records can still change how U processes that content, residual risk 1), and that the response is exactly what U sent (tool calls included). |
| P4 | Verifiable accounting: the charge is computed from the token usage the upstream itself reported for the request, verified by N (who settles) and by C (who checks the signed receipt against its own plaintext). |
| P5 | Upstream authenticity: both parties verify that the connection terminates at a server holding a valid certificate for the upstream hostname. |
| P6 | Freshness: every connection is live and every request on it is bound to one payment; nothing can be reused across connections or requests. |

Non-goals: hiding the prompt from U (U processes it by definition); proving which model weights ran; hiding traffic volume and timing from N (section 5); hiding from U that the requests on one connection belong together (residual risk 16); and third-party verifiability. The proofs convince the two parties of the connection, which is what settlement needs. They are interactive and designated-verifier, and C, holding the server's application key, could forge server records for anyone but N. A receipt is N's signed statement, not a proof to outsiders.

Adversaries: either C or N may deviate arbitrarily from the protocol (they are malicious towards each other); U follows TLS and its own API, and is trusted with the prompt and the API key as it is today. Assumptions: the standard hardness of ECDH over P-256, SHA-256, HMAC and AES-GCM, and two assumptions on the SHA-256 compression function that the split key schedule uses directly (section 4.4); fixed-key AES as a correlation-robust hash (the usual assumption of free-XOR garbling); security of oblivious transfer and correlated-OT extension (KOS, Ferret) against a malicious party in either role (the 2PC relies on security against a malicious receiver, the proofs on security against a malicious sender), and of VOLE-based zero knowledge; the WebPKI. No trusted hardware.

# 3. Preliminaries and the provider profile

## 3.1 Provider profile

The protocol is defined for any upstream satisfying:

1. **TLS 1.3** with an AES-GCM suite in the SHA-256 family (this work: `TLS_AES_128_GCM_SHA256`) and ECDHE over **secp256r1**. No compression, no 0-RTT, no HelloRetryRequest for a P-256 share.
2. **An HTTP framing the protocol knows.** Either **HTTP/1.1** with explicit `Connection: close` and `Accept-Encoding: identity` (framing "1.1"), or **HTTP/2** with the properties H1 to H8 below (framing "2"). The operator lists the framings the upstream supports in the offer.

3. **KeyUpdate accepted** (client-initiated, `update_not_requested`): mid-request on HTTP/1.1, directly after the first flight on HTTP/2.
4. **No processing before the complete body.** The provider starts work only once the whole `Content-Length` body has arrived. N relies on this to stop a request whose suffix does not verify (section 4.6).
5. **Parameter pinning semantics**: a JSON request whose *effective* parameters are fully determined by a public suffix that the client can append. The provider must:
   - keep the last value of duplicate top-level keys;
   - let a later object value replace an earlier one rather than merge with it;
   - reject malformed JSON and unknown fields, so the private prefix cannot leave an object or a string open for the suffix to land in.
6. **Cost-relevant parameters overridable by the suffix**: model, token caps (which must cap reasoning tokens too), sampling count, and provider feature flags that change price, including any injected system prompt. The provider must reject any other price-changing field. The size of the prompt needs no bound, because the node reserves for a full context window (section 4.6).
7. **A streaming (or terminal) usage event** carrying input/output token counts and, ideally, a provider-computed cost figure, emitted before the response ends. The shape the current parser expects is item 11.
8. **No reflection of the request head into the response.** C reads the response, so a provider that echoed the `Authorization` header (for example in an error body) would hand C the key.
9. **Event framing the usage parser expects.** The streamed events use LF line endings only (no CR anywhere in the dechunked body), each event is a single `data:` line of JSON followed by a blank line, and the stream ends in `data: [DONE]`. N's relay and C locate the usage event by this framing, and the argument that R is unambiguous (section 4.7) relies on it.
10. **No copy for the account owner, no forwarding.** The owner of the API key (N) cannot retrieve request or response content from the provider (no request log or stored output readable through the account), and no request parameter, path or header sends it anywhere else (a callback, a webhook, a third-party search). C pins the head template's route to exactly the API base it trusts for that host followed by the endpoint of its kind (a canonical path, no query string) and, in framing "1.1", its headers to a fixed set (`Host`, `Authorization`, `Content-Type`, `Accept-Encoding: identity`, `Connection: close`, `Content-Length`; in framing "2" the header blocks are fixed by the protocol, section 4.6), and an image request's fixed pins may only switch a provider extra off, so N cannot pick a path, header or pin that turns such a feature on. What this item covers is the provider's own behaviour on those requests.
11. **Usage event shape.** These are properties of the case-study provider that the current parser (N's relay and the clients) relies on; a provider without them needs a code change, not a new offer. The response body uses HTTP chunked transfer encoding in framing "1.1" (R carries the chunk framing) and DATA frames in framing "2" (R carries the frame headers). The usage is a separate final event with an empty `choices` array, directly before `data: [DONE]`, with integer `prompt_tokens` and `completion_tokens`. `prompt_tokens` counts every input token, including those read from and those written to the provider's prompt cache: the charge (section 4.8) bills cached and written tokens as parts of the prompt count, so a provider that reported cache writes outside it would be under-billed. The case-study provider's event names the model, but the parser does not need the name, nor that it equal the pinned id: $\pi_{WIN}$ already proved that the request names the pinned model, so N settles at that model's frozen rates, and N's relay logs a different reported name and passes it to N.
12. **Nothing outside the body changes its processing.** No request header or endpoint outside the template changes how U processes the body: routes it to another model or backend, adds a system prompt, changes moderation, caching or retention. N can write a head of its own choosing in epoch 0 without C seeing it (residual risk 1), so this item bounds what such a head can do to C's request.

Items (5) and (6) quantify over every field and input, which no finite test can establish. For the case-study provider (Appendix A) a live regression gate tests items 1 to 7 and 9 with sampled cases, and of item 11 the separate usage event with empty `choices`, its model and the terminator (check A7); the requirements hold as far as those samples show, and the gate is re-run after provider changes. The chunked encoding is not gated (a response without it cannot be disclosed, so the request fails closed), and neither is the cache accounting of item 11. Item 8 is a property of the provider's error handling that the gate does not test, and items 10 and 12 are properties of the provider's account and API design that no request can test; for the case-study provider we found no such retrieval or forwarding (item 10) in its published API on the two routes used, and we have not established item 12 for it. Everything except the specifics of items 4 to 7, 9 and

11 is provider-generic: the epoch structure, the 2PC, the proofs and the record assembly do not change per provider; the head template, the pinned suffix, the price list and the usage parser do. The head template has fixed constraints too: N's relay and the clients accept only `Authorization: Bearer {KEY}` and exactly the six headers of item 10, and $\pi_{NK}$ fixes the key alphabet to `[A-Za-z0-9_-]`, so a provider that authenticates otherwise (another header, a query parameter, a key with other characters) needs a code change. The relay's defaults (upstream `api.venice.ai:443`, the key in `VENICE_API_KEY`, a built-in template on the `/api/v1` path) are the case-study provider's; another provider sets them in the relay's flags and offer file. In the current implementation the head template, the pinned suffix and the price list come from the signed offer (section 4.6); the usage parser expects an OpenAI-style streamed usage event, so a provider with the same streaming shape is a new sidecar offer file plus a profile run, and one with another usage format still needs a code change in the relay and the clients. Image generation (section 4.10) likewise renders its suffix from the signed offer (path, count key, fixed pins, variant pins). A second image provider that has the same shape (a per-image price book with resolution and quality tiers, WebP output, and one of the two response shapes the clients parse) is a new offer entry; a provider priced per megapixel, keyed on other tier parameters, or answering in another shape still needs code in the node and the clients.

**What framing "2" needs.** An operator offers HTTP/2 only when all of these hold for the upstream; `provider-profile-check.py --h2` checks them:

| # | Requirement |
|---|---|
| H1 | TLS 1.3 with ALPN `h2` on the API host |
| H2 | a TLS KeyUpdate is accepted right after the first flight of an HTTP/2 connection |
| H3 | the provider keeps the request dynamic table (default size at least the credential entry) and does not shrink it by SETTINGS |
| H4 | `HEADER_TABLE_SIZE = 0` is honoured for response headers |
| H5 | a request naming `authorization` by index only is authenticated, also after the offer's idle timeout |
| H6 | request bodies up to the node's body limit pass flow control |
| H7 | no response path reflects the `authorization` value (the same list as item 8) |
| H8 | the streamed usage event and `[DONE]` end the stream as section 4.7 parses them |

H3 is also enforced at run time (section 4.5). H7, like item 8, cannot be established by a finite test; the gate samples response paths.

**The image route.** Image generation uses a second route of the same provider, which must satisfy items 1 to 5, 8, 10 and 12 on its own: the transport (TLS 1.3, the one suite, P-256, a client KeyUpdate mid-request), no processing before the complete body, a JSON parser that keeps the last duplicate key, replaces arrays, rejects unknown fields and malformed JSON, and no reflection of the request head. Items 6 and 7 change form. The price of an image is a function of request parameters only (model, count, resolution tier, output format, and paid extras that must be pinned off), so every one of them must be overridable by the suffix; there is no usage event to parse. For the case-study provider's native image route these properties were checked live (`provider-profile-check.py --images`, checks I1 to I9, three paid images; the output is in `docs/evidence/`): a duplicate count and model in the private prefix are overridden by the suffix (I1); unknown fields (I2) and an unknown model (I3, no silent fallback to a default model) are rejected; neither a success nor an error response echoes the request head (I4); a prefix that leaves a string open cannot swallow the suffix (I5); every paid extra and a non-empty `style_references` array are replaced by the pinned values (I6); nothing is sent back before the body is complete (I7); a client offering only TLS 1.3, the one suite and P-256 is served (I8); and a client KeyUpdate in the middle of the body is accepted (I9).

## 3.2 TLS 1.3 key schedule

The TLS 1.3 key schedule (RFC 8446 section 7.1, SHA-256) derives, from the ECDHE shared secret Z and the transcript hashes th_SH (through ServerHello) and th_SF (through server Finished):

$$HS = \text{HKDF-Extract}(\text{Derive-Secret}(ES, \text{"derived"}), Z)$$

$$c_{hs}, s_{hs} = \text{HKDF-Expand-Label}(HS, \text{"c hs traffic" / "s hs traffic"}, th_{SH})$$

$$MS = \text{HKDF-Extract}(\text{Derive-Secret}(HS, \text{"derived"}), 0^{256})$$

$$c_{ap,0}, s_{ap,0} = \text{HKDF-Expand-Label}(MS, \text{"c ap traffic" / "s ap traffic"}, th_{SF})$$

$$c_{ap,i+1} = \text{HKDF-Expand-Label}(c_{ap,i}, \text{"traffic upd"})$$

where ES is a public constant without a PSK. Each traffic secret yields an AES key $K$ and IV through the "key" and "iv" labels. A record with sequence number $n$ uses nonce $IV \oplus n$; its ciphertext is the plaintext XOR the AES-CTR keystream from counter 2, and its tag is $\text{GHASH}_H(AAD, C) \oplus \text{AES}_K(J_0)$ with $H = \text{AES}_K(0^{128})$. Every step is a one-way HMAC-SHA-256, so any set of secrets on the right of a derivation reveals nothing about the secrets that are not derived from it.

HKDF-Extract is one HMAC call, and every HKDF-Expand-Label of the schedule is one HMAC call whose message (length, label, context, a counter byte) is public and fits one SHA-256 block. With $f$ the SHA-256 compression function, HMAC under a key $k$ first computes two states that depend only on the key, $IS_k = f(IV, k \oplus \text{ipad})$ and $OS_k = f(IV, k \oplus \text{opad})$, then an inner digest from $IS_k$ and the message, then the result from $OS_k$ and that digest. Section 4.4 uses this structure.

KeyUpdate: a sender can rotate its own sending keys at any record boundary ($c_{ap,i} \to c_{ap,i+1}$); the receiver updates likewise. Sequence numbers restart in each epoch.

## 3.3 Cryptographic building blocks

**VOLE-based zero knowledge.** We use the interactive QuickSilver-style ZK of TLSNotary's `mpz2`, where SHA-256 (as a compression-function circuit) and AES-128 are available as gadgets. Both parties are online during the connection, so interactivity costs only round trips. Every plaintext comparison in our statements is done inside the circuit and only a single equality bit is decoded.

**Garbled circuits.** A garbler encodes a Boolean circuit so that an evaluator who holds one label per input wire learns the output and nothing else. With free XOR, the two labels of every wire differ by a global offset $\Delta$ that only the garbler knows, and XOR gates cost nothing. With point-and-permute, the last bit of the label the evaluator holds is a pointer bit $\Lambda_w = z_w \oplus \lambda_w$: the wire's true value $z_w$ hidden by a mask $\lambda_w$ that the garbler chose. Half-gates garbling (Zahur, Rosulek and Evans 2015) sends 256 bits per AND gate; three-halves garbling (Rosulek and Roy 2021) splits labels into halves and sends $1.5\kappa + 5 = 197$ bits per AND gate. Plain garbling protects the garbler's inputs from the evaluator, but nothing stops a malicious garbler from garbling a different circuit.

**IT-MACs and authenticated garbling.** An information-theoretic MAC on a bit $x$ held by one party gives that party $x$ and $M = K \oplus x\Delta_B$ over $GF(2^{128})$, and gives the other party the key $K$ and a global key $\Delta_B$. The holder can open $x$ only to its true value unless it guesses $\Delta_B$, and MACs are linear, so XORs of authenticated bits are authenticated for free. Random correlated oblivious transfer (KOS, or Ferret on top of it) produces such MACs cheaply, and a QuickSilver-style check verifies a whole batch of products $t = x_i x_j$ at the cost of two field elements. *Authenticated garbling* (Wang, Ranellucci and Katz 2017, and its successors) uses IT-MACs on the wire masks to make a single garbled circuit secure against a malicious garbler. In the full constructions the masks are secret-shared between the two parties, which removes all leakage but needs authenticated AND triples whose generation costs more bandwidth than garbling twice. We use a garbler-only form (section 4.4): the garbler keeps the masks, authenticates them to the evaluator, and the evaluator checks every AND gate with one batched check. The price is a few bits of selective-failure leakage per connection.

**Share conversion (ECtF).** Two parties holding ECDH contributions can convert the two halves of a point sum into additive shares of its x-coordinate with a few multiplicative-to-additive (M2A) and additive-to-multiplicative (A2M)

conversions over the base field, built on oblivious linear evaluation. TLSNotary uses the same construction for its key exchange.

**HTTP/2 and HPACK.** HTTP/2 (RFC 9113) multiplexes requests as *streams* of typed *frames* on one connection; each frame has a 9-byte header (length, type, flags, stream id). A request is a HEADERS frame followed by DATA frames. Header fields are compressed with HPACK (RFC 7541): each endpoint keeps, per direction, a *dynamic table* of (name, value) entries that the peer's encoder has asked it to store. A field is sent as an index into the static or dynamic table (the whole pair), as a literal whose name is taken from a table entry, or as a full literal; a literal "with incremental indexing" is also appended to the table. The first dynamic entry has index 62. HPACK was designed after the CRIME attack on DEFLATE-compressed headers: it has no substring matching, only whole-field references.

# 4. Protocol

The unit of the protocol is a **connection**: one TLS 1.3 connection to U, opened by one two-party handshake and identified by a connection id `sid`. A connection carries **requests**, numbered $n = 1, 2, ...$ and named $(sid, n)$. How many it carries depends on the HTTP **framing** that C picks from N's signed offer (section 4.2): with HTTP/2 a connection carries many requests one after another, with HTTP/1.1 exactly one. Sections 4.1 to 4.5 describe what happens once per connection, sections 4.6 to 4.9 what happens once per request.

## 4.1 Who holds which secret

The protocol is built around a split of the TLS secrets. C and N jointly emulate one TLS client; the 2PC releases to each party only what it needs:

| Value | C | N | Purpose |
|---|---|---|---|
| ECDHE scalar, Z | additive share | additive share | key exchange |
| HS, the derived secret, MS, $c_{ap,0}$ and their outer HMAC states | never | never | inside the 2PC only |
| the inner HMAC states of those four keys | yes | yes | public; both compute the inner digests (section 4.4) |
| $c_{hs}$, $s_{hs}$ | yes | yes | both verify the handshake (P5) |
| $K_0$, $IV_0$ (from $c_{ap,0}$) | **never** | yes | N's credential record (P1) |
| $c_{ap,1}$ | yes | **never** | C's requests (P2) |
| $s_{ap,0}$ | yes | **never** | the responses (P2) |
| $h_1 = \text{SHA256}(c_{ap,1})$, $h_2 = \text{SHA256}(s_{ap,0})$, $h_k = \text{SHA256}(K_0 \| IV_0)$ | yes | yes | public anchors for the proofs |

$c_{ap,1}$ is derived from $c_{ap,0}$, but not the other way round, so C's body key does not reveal N's record key. Neither $K_0$ nor $IV_0$ yields $c_{ap,0}$. U sees an ordinary TLS client.

## 4.2 Connections, framings and epochs

| Epoch | Key | Writer | Content |
|---|---|---|---|
| Handshake | $c_{hs}$ / $s_{hs}$ | C (U) | ClientHello with the joint key share; client Finished computed by C from $c_{hs}$ |

| Epoch | Key | Writer | Content |
|---|---|---|---|
| 0 | $c_{ap,0}$ ($K_0$, $IV_0$) | **N** | sequence 0: the **connection head**, which carries the credential; sequence 1: `KeyUpdate(update_not_requested)` |
| 1 | $c_{ap,1}$ | C | every request; N releases C's records to U only as $\pi_{WIN}$ allows (section 4.6) |
| Response | $s_{ap,0}$ | U | responses; N relays ciphertext |

C writes nothing in epoch 0 and starts epoch 1 at sequence 0. An honest N writes exactly two records, in one write (a dishonest one can write more, residual risk 1). U processes epoch 0 as an ordinary client's first records followed by a KeyUpdate, and decrypts everything after it under $c_{ap,1}$. C never sends a KeyUpdate of its own.

The credential has to be written under $K_0$, which only N holds, and a garbled key schedule is single-use. So the credential can be written once per two-party handshake, and the question is how many requests that one write can serve. The two framings answer it differently.

**Framing "2" (HTTP/2).** The connection head is the HTTP/2 client preface, a SETTINGS frame, and one cheap authenticated **probe request** whose `authorization` header is encoded as an HPACK *literal with incremental indexing*. The provider's HPACK decoder therefore stores the header in its dynamic table for this connection, and it is the only entry: index 62. Every later request is written by C under $c_{ap,1}$ and names the credential by that index, a single byte `0xbe`, without knowing its value. One two-party handshake then serves up to `max_requests` sequential requests (an offer parameter, 100 by default) within `max_lifetime_s` (3600 s by default). Section 4.5 gives the head, section 4.6 the rules that keep C's records to what they must be, and section 5 the argument that the stored value cannot be read back.

**Framing "1.1" (HTTP/1.1).** The connection head is the request head of the connection's only request, with the credential in its `Authorization` line, `Content-Length` and `Connection: close`. The body follows in epoch 1 and the provider closes the connection after the response. An HTTP/1.1 head names the path and the body length, so it belongs to a request: with this framing C sends its first (and only) `request` before the ClientHello, so that N knows the request before it commits to the head. The HTTP/2 connection head names no request.

**Choosing the framing.** The operator lists in the relay's offer file the framings the upstream supports, in the operator's order of preference (`"http": ["2", "1.1"]`); nothing is probed or assumed, and the list is part of the signed offer. C takes the first entry it supports. If a connection with framing "2" fails before the body of its first request was released (handshake, ALPN, the connection head, the provider closing), C marks that framing bad for this node for ten minutes and serves the request with framing "1.1" on the same node, when the offer lists it. After a body was released there is no retry (section 4.9). N's relay stops offering "2" for ten minutes after five consecutive HTTP/2 connections failed before their first response began, which protects clients from a provider that changed its behaviour. The framing is a property of the inner TLS connection only: the application talks to the client daemon as before and never sees it.

One state machine serves both framings:

| State | Framing "2" | Framing "1.1" |
|---|---|---|
| armed (preprocessing and key share done) | waits for the ClientHello | waits for request 1, then the ClientHello |
| connection head (epoch 0, written by N) | preface, SETTINGS, WINDOW_UPDATE, the probe request with the credential, RST_STREAM | the request head of request 1 with the credential |

| State | Framing "2" | Framing "1.1" |
|---|---|---|
| ready (after the head is written) | accepts request $k$ | request 1 is already open; its records follow |
| after a settled 2xx | ready again | closed |
| requests per connection | up to `max_requests`, sequential | exactly 1 |

## 4.3 Setup, preprocessing and the joint key share

**Setup.** C opens a control channel to N and sends the framing it chose, a nonce, its ordinary node credential and the signature of the offer it verified. The setup names no request: no model, no token cap, no body length. N checks that the framing is offered and that it signed and still remembers the offer, validates the credential, and reserves the **connection deposit** on it (section 4.8). Only then does it dial its relay process. A client without funds gets no garbled circuit, and N opens no connection to U at this point.

**Preprocessing.** Before any TLS byte is sent, N garbles the key-schedule circuit (section 4.4) and streams the tables to C, the parties authenticate N's wire masks to C and run the checks on them, and they run the oblivious-transfer setup for C's input bits and the random OLEs for the share conversion. This step is independent of all inputs. It moves 16.4 MB from N to C and 1.1 MB from C to N, and it cannot be reused: a garbled circuit is single-use, and each connection needs one handshake and one credential record.

**Joint key share.** C draws a scalar $a_C$, sends a commitment SHA256$(P_C \| \rho)$ to $P_C = a_C G$, receives $P_N = a_N G$ from N and opens. Both compute $P = P_C + P_N$. Committing first prevents C from choosing $P_C = T - P_N$ for a point $T$ whose discrete logarithm it knows. Scalars are fresh in every connection. The key exchange uses P-256 rather than X25519 because the joint share needs full point addition, which X25519's x-only Montgomery form does not offer.

**ClientHello enforcement.** C builds a ClientHello with exactly one key share, the joint point $P$, on secp256r1; `supported_groups` lists only secp256r1; one cipher suite; no PSK and no early data. **N parses the ClientHello and relays it only if all of this holds.** Without this check C could add a share of its own, finish the handshake alone and learn every secret. N rejects a HelloRetryRequest and checks that the ServerHello selects secp256r1 and the offered suite, then extracts U's share $S$. N also fixes the ALPN extension: exactly `h2` for framing "2", and N requires U's EncryptedExtensions to select it; `http/1.1` or none for framing "1.1".

**Armed connections.** A setup may ask N to stop after the key share. Such an *armed* connection holds the deposit and the preprocessing but no connection to the provider (N's relay dials U only when the ClientHello arrives), and lives at most ten minutes. It is bound to the credential only, so any request can use it. This is all that remains of v2's pre-warming (section 6.3).

## 4.4 The two-party key schedule

**ECtF.** C computes $a_C S$ and N computes $a_N S$. A point-addition share conversion (two A2M and one M2A over the P-256 base field) turns these into additive shares $x_C + x_N = x(a_C S + a_N S) \bmod p$ of the ECDHE secret Z, without either party learning Z.

**Splitting HMAC at the compression function.** The key schedule is a chain of HMAC-SHA-256 calls. Written with the SHA-256 compression function $f$,

$$\text{HMAC}(k, m) = f(OS_k,\ \text{pad}(f(IS_k, m))), \qquad IS_k = f(IV, k \oplus \text{ipad}), \quad OS_k = f(IV, k \oplus \text{opad}),$$

for a message $m$ that fits one block after the key block, which holds for every call of the TLS 1.3 schedule. Every message of the schedule is public: a label, a transcript hash, a constant. Only four keys are secret: HS, the "derived" secret $dHS$, MS and $c_{ap,0}$. The circuit therefore computes, for each of these keys, the two pad states $IS_k$ and $OS_k$, and then only the **outer** compressions. The inner state $IS_k$ is a **public output**: both parties compute the inner digest $f(IS_k, m)$ in the clear, N sends the labels of the digest's input wires, and C pins them by pointer checks against opened, authenticated masks. $OS_k$ never leaves the circuit. The protocol calls this key schedule `split-public-1`; the offer names it, and the first message of the 2PC channel in each direction is a version tag.

**Stages.** It is one circuit, evaluated in five stages. C runs a consistency check on each stage before it uses or reveals any output of that stage, and each stage's public output must reach N before N can send the digest labels of the next.

| Stage | Garbled | Opened |
|---|---|---|
| A1 | $Z = x_C + x_N \bmod p$; $HS = \text{HMAC}(salt, Z)$; $IS_{HS}$, $OS_{HS}$ | $IS_{HS}$ (public) |
| A2 | $c_{hs}$, $s_{hs}$, $dHS$ from $OS_{HS}$; $IS_{dHS}$, $OS_{dHS}$ | $c_{hs}$, $s_{hs}$, $IS_{dHS}$ (public) |
| A3 | MS from $OS_{dHS}$; $IS_{MS}$, $OS_{MS}$ | $IS_{MS}$ (public) |
| B1 | $c_{ap,0}$, $s_{ap,0}$ from $OS_{MS}$; $IS_{c0}$, $OS_{c0}$ | $s_{ap,0}$ (to C), $IS_{c0}$ (public) |
| B2 | $K_0$, $IV_0$, $c_{ap,1}$ from $OS_{c0}$; $h_1$, $h_2$, $h_k$ | $c_{ap,1}$ (to C), $K_0 \| IV_0$ (to N), the anchors (public) |

Stages A1 to A3 run after the ServerHello and must finish before U gives up waiting for the client Finished; B1 and B2 run after the server Finished. The circuit has 479,022 AND gates and 22 SHA-256 compressions, against 656,179 and 31 for a key schedule garbled in full (section 7.1). The price is round trips: the handshake part takes 3 and the application part 1.5.

**Handshake verification.** With $s_{hs}$, C decrypts U's encrypted handshake messages in its own TLS stack and verifies the certificate chain to the WebPKI roots, the hostname, CertificateVerify and the server Finished, then computes and sends its Finished from $c_{hs}$. N independently decrypts the same messages, reconstructs the transcript and verifies the chain, hostname, CertificateVerify and both Finished messages, fail-closed (P5). A wrong Z, for example after a deviation in ECtF, fails the server Finished here.

**Garbling and authenticated masks.** Only N garbles. In preprocessing, N garbles the circuit once with three-halves garbling under its global offset $\Delta_A$. The mask $\lambda_w$ of every wire is the pointer bit of N's label for the value 0, and C only ever sees the pointer bits $\Lambda_w = z_w \oplus \lambda_w$. C draws a global MAC key $\Delta_B$ (128 bits, 127 of them random). Using random correlated OT in which C is the sender (Ferret, bootstrapped by KOS), N authenticates to C the mask of every circuit input and every AND output (the *base* wires) and, for every AND gate $k$ with inputs $i, j$, the product $t_k = \lambda_i \lambda_j$. The masks of all other wires follow by linearity. N sends its tables and these commitments first; only then does C send a 32-byte seed, from which both derive the challenges and a per-connection salt. N then proves all products $t_k$ with one QuickSilver-style check, opens the masks of C's input wires, of the digest input wires and of the outputs C decodes (the bits and one hash of their MACs), and sends, per stage, one check row per base wire and a value $Y_N$.

**Evaluation and the consistency check.** Online, C obtains the labels of its inputs by oblivious transfer, and N sends the labels of its own inputs and of each stage's digest. C checks the pointer bits of its own inputs and of the digest against the opened masks, so N can neither substitute C's inputs nor feed in another digest. C evaluates the stage and then checks all its AND gates at once. Gate $k = (i, j \to o)$ was evaluated consistently with the committed masks if and only if

$$e_k = \lambda_o \oplus t_k \oplus \Lambda_i \lambda_j \oplus \Lambda_j \lambda_i \oplus \Lambda_o \oplus \Lambda_i \Lambda_j = 0.$$

Under the MACs, $\sum_k \chi^k e_k = 0$ becomes a linear equation between $Y_N$ and the check rows on one side and, on the other, a value C computes from its MAC keys and its active labels. N does not know the pointer bits, so each row is padded with hashes of the wire's two labels and only the label C actually holds opens it. C accepts if and only if the two sides agree. The rows and $Y_N$ are sent truncated to 64 bits, while all arithmetic stays in $GF(2^{128})$. The hash in each row is tweaked by the salt, which N learns only after it has committed its tables, its offset and its keys (section 5 explains why this matters).

**Outputs and label-check pairs.** C decodes its own outputs and the public ones with the authenticated masks. For every output wire that C reports to N (the public outputs and N's private ones), N sent in preprocessing the salted hashes of both candidate labels. C checks the label it holds against that pair before it reveals anything, and aborts on a mismatch. N decodes its outputs from the reported labels with its own keys. C cannot change N's outputs, because that needs an inactive output label, and inactive labels need $\Delta_A$. A stage's three checks (input pointer bits, consistency,

label-check pairs) end in one byte to N, sent after C has evaluated the whole stage, so an abort looks the same whatever failed. **No output is sent to U, used to encrypt, or revealed before the checks of its stage pass.**

**What a cheating party gains.** A cheating N can garble a different circuit, lie about a mask, a product or an opened bit, corrupt check rows or label-check pairs, send a wrong digest label, or run the oblivious transfer for C's inputs with a wrong correlation. Each of these either fails the product check, the openings, the pointer checks, the consistency check or a label-check pair, or leaves C's outputs correct (section 5). What N can still do is make an abort depend on C's private values (selective failure): the abort can be any predicate of C's inputs and of the circuit's internal wire values, which include C's own secrets $s_{ap,0}$ and $c_{ap,1}$. N observes one of eight outcomes per connection: an abort in ECtF (C's degenerate-share check, which a deviating N may be able to make depend on C's input), an abort in one of the five stages, the server Finished failing after a shifted Z, or success. The connection ends at the first failure. So N learns at most $\log_2 8 = 3$ bits per connection of additional selective-failure leakage, in this idealized abort model (every deviation ends in one of these outcomes and N observes nothing else), provided ECtF reveals nothing beyond its outcome (below) and the garbling argument holds. N also receives the outputs it is meant to have; the bound is about the rest. A proof would have to enumerate every distinguishable result of a malicious ECtF, OT and preprocessing, of N's own report checks, of the TLS verification, of timeouts and errors; that has not been done. A cheating C, the evaluator, sees the tables, its active labels, uniformly random pointer bits, the opened masks of values it already knows or decodes, check rows and label-check pairs padded by hashes of labels it does not hold, the public states $IS_k$, and $Y_N$, which it can compute itself. It learns nothing beyond its outputs and the public states and can only choose its own inputs, as in any 2PC. ECtF uses `mpz2`'s OLE-based A2M and M2A, which are secure against semi-honest parties only. We do not argue that they stay private against a malicious N; the bound above is conditional on that.

**What the public states cost.** Both parties see $IS_k$ for the four secret keys. With the key schedule garbled in full, the security of the outputs rests on HMAC-SHA-256 being a PRF. Here it rests on two statements about the compression function: (1) $f$ keyed through its chaining input is a PRF, and (2) $OS_k$ is pseudorandom given $IS_k$, that is, $f$ keyed through its message input by $k \oplus$ ipad and $k \oplus$ opad gives unrelated outputs. These are the two assumptions of Bellare's proof that HMAC is a PRF (CRYPTO 2006); we use them directly instead of through the theorem, and for both parties, since both see $IS_k$. No output secret is derivable from an $IS_k$ without $OS_k$: every derived value passes through an outer compression keyed by $OS_k$. Every inner digest moved out of the circuit has a public message (a fixed label and a transcript hash); the one HMAC with a secret message, $HS = \text{HMAC}(salt, Z)$, stays inside.

A third requirement comes from the exposed inner state: (3) **the inner digests of one key are pairwise distinct.** Several outputs are derived under one $OS_k$, and equal inner digests would give equal outputs: $c_{ap,0} = f(OS_{MS}, \text{pad}(d_{c0}))$ and $s_{ap,0} = f(OS_{MS}, \text{pad}(d_{s0}))$, so $d_{c0} = d_{s0}$ would hand C, who holds $s_{ap,0}$, the key $c_{ap,0}$ and with it $K_0$; a "derived" digest equal to $d_c$ or $d_s$ under $OS_{HS}$ would expose the derived secret and MS. The messages of one key differ in their fixed HKDF labels at fixed positions of the block, and $IS_k$ is known, so an equal pair is a collision of the SHA-256 compression function under a known, unchosen chaining value (generic cost about $2^{128}$). With the inner key secret this was covered by the PRF assumption; here it is an assumption of its own. Bellare's theorem is about one key with hidden pad states; the chained, multi-output system with public $IS_k$, outputs for different parties, public anchors and selective failure needs its own hybrid argument, which has not been written.

**Why the leakage cannot accumulate.** The bits N can learn are about that connection's values: C's ECDHE scalar share and the key schedule's internal wires, including C's traffic secrets $s_{ap,0}$ and $c_{ap,1}$. The protocol keeps one invariant: **no 2PC input and no garbling key is ever reused across connections.** Scalars, shares, labels, $\Delta_A$, $\Delta_B$, the MAC keys and the salt are all drawn fresh for each connection and used once, and an aborted connection is discarded; the traffic secrets are fresh because the ECDHE secret is. Bits leaked in different connections are therefore about independent values and never add up to any secret, and 3 bits of a 256-bit traffic secret leave it out of reach. The API key is never an input to the 2PC: it exists only in N's epoch-0 record. A connection that carries many requests exposes all of them to the same 3 bits, not 3 bits each.

**Inputs are never opened.** Every private input enters only as a garbled input or an OLE input, and nothing is revealed after the connection. This distinguishes the construction from 2PC-TLS designs in which one party opens its shares afterwards (section 9), which N cannot do here: its shares would give C the master secret and therefore $K_0$.

What each party learns, in total:

| Party | Learns | Does not learn |
|---|---|---|
| C | $c_{hs}$, $s_{hs}$, $s_{ap,0}$, $c_{ap,1}$, $h_1$, $h_2$, $h_k$, the four $IS_k$; whether a check passed | Z, HS, MS, $c_{ap,0}$, any $OS_k$, $K_0$, $IV_0$, the API key |
| N | $c_{hs}$, $s_{hs}$, $K_0$, $IV_0$, $h_1$, $h_2$, $h_k$, the four $IS_k$; whether a check passed; per request the public windows and the usage event (as one-bit-checked public values) | Z, HS, MS, any $OS_k$, $s_{ap,0}$, $c_{ap,1}$, prompt, response |
| U | an ordinary TLS 1.3 connection | that two parties are involved |

## 4.5 Connection head (epoch 0) and $\pi_{NK}$

After stage B2 and the handshake verification, N builds the connection head of the framing, encrypts it under $K_0$ at sequence 0, and seals `KeyUpdate` at sequence 1. It sends C a commitment $c = \text{SHA256}(header \| C_1)$ to the ciphertext and proves its content.

**The head of framing "2"** is one record:

```
"PRI * HTTP/2.0\r\n\r\nSM\r\n\r\n"
SETTINGS:       HEADER_TABLE_SIZE = 0, ENABLE_PUSH = 0, INITIAL_WINDOW_SIZE = 2^31-1
WINDOW_UPDATE:  stream 0
HEADERS stream 1 (END_STREAM|END_HEADERS), no Huffman coding:
    82  87  01 <len> <host>  04 <len> <probe_path>  57 <len> "Bearer " <KEY>
RST_STREAM stream 1 (CANCEL)
```

The HEADERS frame is a `GET` of a cheap authenticated path of the provider (the offer's `probe_path`, which C accepts only if it is the trusted API base followed by `/models`). The byte `0x57` means "literal header field with incremental indexing, name from static index 23 (`authorization`)". After this block the provider's request table holds one entry, `authorization: Bearer <KEY>`, at index 62. `HEADER_TABLE_SIZE = 0` tells the provider not to index anything in the *response* direction, and `ENABLE_PUSH = 0` disables server push. The probe is reset at once; its only purpose is to carry the header. The head has a fixed length and the key sits at fixed offsets.

**The head of framing "1.1"** is the offer's head template for the request's kind (the chat or the image path), with `Content-Length` set to the declared body length, `Connection: close` and the credential.

$\pi_{NK}$ **(N to C)**, once per connection.

- *private*: $K_0$, $IV_0$, the key bytes;
- *public*: $h_k$, the template with the key positions, the key alphabet, $|head|$, the record header, the commitment $c$;
- *statement*: $\text{SHA256}(K_0 \| IV_0) = h_k$; the AES-CTR keystream under $K_0$ with nonce $IV_0 \oplus 0$; $C_1 = (head \| \texttt{0x17}) \oplus$ keystream; $\text{SHA256}(header \| C_1) = c$; $head$ equals the template outside the key positions; every key byte is in the alphabet. Only the digests and the alphabet bit are decoded.

C rebuilds every public input from the signed offer, the framing, for framing "1.1" its own body length, and the $h_k$ that its own 2PC output, so N cannot prove a different template or a key other than the one the 2PC produced. The proof binds the head's content and length. It does not bind the bytes N actually writes: C never sees $C_1$, so a different head of the same length is not detectable by C (section 5, residual risk 1).

When $\pi_{NK}$ has verified, C releases the head. N writes the head record and the KeyUpdate in one write and tells C. C sends no record between its Finished and that message. In framing "2" C reports the SETTINGS values U sent; N ends the connection if U's `HEADER_TABLE_SIZE` is below the default 4096: below the credential entry's size the provider would not keep it, and any smaller table makes its decoder expect a table size update that the pinned request block cannot carry. A C that lies here only harms itself: its next request is unauthenticated.

## 4.6 Requests (epoch 1) and $\pi_{WIN}$

**The request message.** C announces request $n$ on the control channel: its kind (chat or images), the model, the token cap (or, for images, the variant and count), the body length, its node credential (which may differ from the setup's), the signature of the offer whose rates it accepts, and the fees. N does what the setup of a one-request protocol would do: it checks that the model is served confidentially and priced by the named offer, validates the credential, and **reserves** for the request (below). The named offer's connection-level fields must equal those of the offer the connection was opened on; otherwise C opens a new connection. N answers with the canonical suffix and, in framing "2", the stream id $2n + 1$. A refused request (no balance, an unknown model) leaves the connection ready when C has sent no record for it, so a 402 does not cost a handshake. One request is open at a time.

**The suffix.** The body is body_private ‖ suffix. body_private opens the JSON object and may carry messages, tools and other schema-valid fields. The suffix is the public string that pins every cost-relevant parameter: model, token caps, `n`, streaming, usage reporting and provider feature flags. The signed offer describes it: the request keys that carry the token cap, then fixed pins in order. N's relay renders it from that description, the model and the token cap declared in the request, and accepts no other; C renders the same bytes from the signed offer. Because N chooses the description, C accepts it only within a narrow policy: the cap keys are among `max_tokens`, `max_completion_tokens` and `max_output_tokens`; `stream` is pinned on with usage reporting (the usage proof needs it) and `n` to 1; every other pin switches something off (`false`, `"off"`, an empty list, or an object of those); no pin key may start with a negation such as `disable` or `no_` (where `false` turns a feature on) or name a protection such as `safe`, `moderat` or `filter` (pinning it could only switch the protection off); values are booleans, integers, plain ASCII strings, lists and objects, rendered as compact JSON with sorted members. The "switches something off" rule is a heuristic over key names and values, not a guarantee: whether `false` turns a feature off depends on the provider's semantics, and a key whose name inverts its meaning without one of the listed prefixes (`logging_opt_out: false`, say) would pass it. The policy narrows what a dishonest offer can pin; the client's user still has to trust the provider's documented meaning of each pinned key. A pin cannot inject content into C's request: a key the offer pins that C's caller also set to another value makes C refuse the request instead of letting the suffix silently replace it. The provider resolves the effective values to the suffix (section 3.1, item 5).

**What C may write.** N cannot read C's records, so everything N needs to know about them is proven. In framing "1.1" this is one fact, as before: the record that completes the body is the suffix. N forwards the earlier body records, enforces that the epoch-1 plaintext is exactly the declared length, and holds the final record until the proof verifies. Until then U has an incomplete request and does nothing (section 3.1, item 4).

In framing "2" C writes HTTP/2 frames, and a frame is an instruction to U's HTTP/2 stack, not only content. A HEADERS frame of C's choosing could name another path, add a header, or open a second stream that N does not bill. The rules are:

1. **One frame per record.** The inner plaintext of every record C writes is a 9-byte frame header, the payload, and the content type byte `0x17`, with no TLS padding.
2. **N writes a record to U only after a proof that covers it has verified.**
3. For each record the proof shows that bytes $[0, 9)$ equal the frame header N expects and that the **last byte is `0x17`**; the header's length field equals the record's inner length minus 10. Without the last-byte check C could shorten the declared frame with TLS padding in its place, so that U's parser found the next frame boundary inside a later payload, and hide a HEADERS frame there.
4. **Allowed frames.** For a request: one HEADERS frame with END_HEADERS, whose whole record is proven equal to a public block, and DATA frames, with END_STREAM only on the last, whose tail is proven equal to the suffix. Between and around them: SETTINGS acknowledgements, PING acknowledgements, WINDOW_UPDATE on stream 0, and RST_STREAM on the current request's stream, each announced on the control channel and proven in full. Anything else ends the connection.
5. C never sends a KeyUpdate, a header block that adds a dynamic-table entry, a table size update, or a second stream while one is open.

The request's header block is public and both sides build it from the offer:

```
83  87  01 <len> <host>  04 <len> <path>  0f 10 10 "application/json"  be
```

that is `POST`, `https`, the authority, the chat or image path, the content type, and `be`: indexed field 62, the credential. C

follows U's flow control and tells N up to which record it may write; N never writes an unproven record nor beyond that mark. The body may span many DATA records (a 200 kB body took 13).

$\pi_{WIN}$ **(C to N, before U can act on the records).** One statement covers both framings and replaces v2's suffix proof.

- *private*: a traffic secret ($c_{ap,1}$ for C's records; $s_{ap,0}$ for an error head, section 4.7);
- *public*: the anchor ($h_1$ or $h_2$), the number of key updates since that secret, and a list of **windows**, each naming a record's sequence number, a block-aligned stretch of its ciphertext, an offset in it, and the expected plaintext bytes;
- *statement*: SHA256$(secret) = anchor$; derive the key and IV; for every window, the AES-CTR decryption of the ciphertext at the given position equals the expected bytes. All windows are compared in one equality, and the verifier decodes one bit.

N takes every ciphertext from its own relay buffer, never from C. In framing "1.1" there is one window: the whole final record equals the suffix followed by `0x17`. In framing "2" a request's batch has the HEADERS record in full, the first nine bytes and the last byte of every DATA record, and the tail of the last DATA record (suffix $\|$ `0x17`). A proof holds at most 2,200 windows and 64 KiB of ciphertext, so a long body is proven in several consecutive, disjoint batches while N buffers it, and none of its records is written before the last batch verified: every body the record rules allow (8 MiB in up to $3 + \lceil \text{body}/4096 \rceil$ records) is provable, and the statement "no inference ran on a failed proof" holds for long bodies too. The statement costs 7 SHA-256 compressions plus one AES block per distinct 16-byte block a window touches, since the anchor is a hash of a secret that the 2PC itself output and the key schedule is not recomputed.

**Parameter validation by C.** The public parameters of $\pi_{WIN}$ and $\pi_{C3S}$ come from N. C checks each against its own view before proving: every window must lie in a range that is public for that record by the rules above (a frame header, a last byte, the header block, the suffix tail), must name ciphertext bytes exactly as C sent them, and every record of the batch must be named; $\pi_{C3S}$ (section 4.7) must cover server records exactly as C received them, with C's own disclosure as the expected value; the anchors must equal C's own 2PC outputs. N therefore cannot aim a proof, or its one-bit result, at private plaintext such as the prompt.

When the proof has verified, N writes the held records to U and tells C that the body is released. From here the reservation is at stake. If the proof fails, N closes the connection: no inference ran. A client therefore cannot run a different model or a larger cap than N reserved for, cannot reach another path, and cannot open a stream N does not know about.

**Reservation.** The prompt is opaque to N, so the reservation never discounts it: it is $\max(m, \lceil (L \cdot r_{in} + (T + M) \cdot r_{out})/1000 \rceil)$ msat, where $L$ is the model's context length, $T$ the pinned token cap, $r_{in}, r_{out}$ the per-1k rates of the frozen offer, $m$ the node's minimum charge and $M$ a completion margin (16 tokens by default, an operator setting, published in the signed offer as `reserve_margin_tokens` so C can size its balance). The margin exists because a provider may bill a few tokens beyond the cap (Appendix A) and the charge is capped at the reservation; it only enlarges the hold, and the charge is still the reported usage. Overshoot beyond $M$ is N's loss. (Two node configurations fall outside this formula: a model without a known context length is reserved at its maximum per-request cost plus the margin, and a node with fixed per-request pricing does not offer confidential mode at all, since a usage-based charge has no meaning there.) $L$ must be the whole context, not the context minus the model's own completion limit: the pinned cap $T$ is usually far below that limit, the provider accepts a prompt of up to $L - T$ tokens, and since the charge is capped at the reservation, any prompt tokens beyond the reserved bound would be free. For the case study, `deepseek-v4-flash` (a 1M-token context) with $T = 40$ reserved 167,308 msat for a v2 session that cost 6 (local node, 2026-10-19, `docs/evidence/adversarial-2026-10-19/chat/control.out`). Where the node's catalogue gives the model two context lengths (its own and a top provider's), the larger is reserved.

## 4.7 Response and usage disclosure ($\pi_{C3S}$)

U's response streams under $s_{ap,0}$; C decrypts it and verifies every GCM tag natively (P3 for the response) and streams tokens to the application live. When a request ends, N signs an **attestation** for it, binding the connection id and the request number, the certificate chain hash, the two handshake transcript hashes (through ServerHello and through the server Finished), the record counts and, in framing "2", the stream id and the index $S_k$ of the first server record of the request's interval, the SHA-256 of the bytes sent to U and received from it, the commitment to the head record

($c_{commit}$) and the time. N's relay signs it with a per-process Ed25519 key and N signs it again with its Nostr key; C checks N's signature and that the connection id, the request number and both transcript hashes are its own.

C then discloses R: the plaintext from the boundary of the last usage-bearing data event through the end of the response, with its framing.

$\pi_{C3S}$ **(C to N)**, unchanged from v2.

- *private*: $s_{ap,0}$;
- *public*: $h_2$, the covered records' ciphertexts and sequence numbers, R, a count `skip` of undisclosed bytes after R, and the number $k$ of server KeyUpdates before the covered records;
- *statement*: SHA256$(s_{ap,0}) = h_2$; derive $s_{ap,k}$ by $k$ "traffic upd" steps and its key and IV; CTR-decrypt the covered records; each record's last plaintext byte (its inner content type) is `0x17`, so R cannot be read out of a handshake or alert record or a padded one; R ends `skip` bytes before the end of the covered records; in-circuit equality of R and the content types with their expected values, decoded as one bit. For a usage disclosure `skip` is 0.

**Anchoring in framing "1.1".** The provider closes the connection after the response, so R is anchored at the end of everything N relayed, as in v2.

**Anchoring in framing "2".** The connection stays open, so "the end of the transcript" is not available, and an earlier request's usage event lies in the same transcript. N records $S_k$, the number of server records it had relayed when it wrote the first record of request $k$. The disclosure must lie in the request's **interval**, the server records with index at least $S_k$, and must run to the last server byte N has received. N parses R as alternating text and frames: the DATA payloads on the request's stream must be exactly `data: <usage JSON>`, a blank line, `data: [DONE]` and a blank line, and the stream must end (END_STREAM) inside R. A usage event from request $k-1$ lies before $S_k$ and is refused; connection-level frames inside R are visible to N as frames and carry no content. Frames of the probe stream (stream 1, reset in the head), which U may still answer late, are skipped whole, their payloads kept out of the event text, so billing does not depend on when that answer arrives; a frame of any other stream is refused.

U may rotate its keys mid-response (a KeyUpdate inside the encrypted stream, which only C can see). C then reports $k$ and the record where that epoch starts, whose sequence numbers restart at 0; a wrong report only makes the records decrypt to noise, which cannot equal a well-formed R. Each step costs four compressions; $k$ is at most 16, and U normally sends none. Two rarer cases make the statement unprovable, so a completed request is charged the reservation: a KeyUpdate *between* the covered records (the proof uses one epoch for all of them), and a record after the terminal chunk that is longer than a KeyUpdate or close_notify (e.g. a late NewSessionTicket). U normally sends neither.

The choice of R is unambiguous: with the provider's LF-only, one-line-per-event framing (section 3.1, item 9), and since JSON strings cannot contain raw CR/LF, event boundaries and HTTP chunk framing cannot be forged from model output. We checked this by prompting the model to emit fake usage events; every imitation stayed escaped inside a string (Appendix A). In framing "2" the event framing is carried in DATA frames instead of HTTP chunks, and the same argument applies to frame headers: a 9-byte frame header inside R is checked by N against the lengths it parses, and model output is confined to JSON strings inside DATA payloads.

**Error heads.** An HTTP-level error from U (for example 503, 429, or a 400 for a request U rejects) carries no usage event. C then discloses only the response head and the error body stays private, since it can quote the request. In framing "1.1" this is the status line and header lines, proven by $\pi_{C3S}$ with `skip` set to the length of the rest of the response. In framing "2" it is one complete HEADERS frame of the request's stream in a server record of the request's interval, disclosed from byte 0 of that record: the frames before it in the record, which may only be stream-0 control frames, WINDOW_UPDATEs and frames of the probe stream (none carries C's bytes), are disclosed with it. Walking them from byte 0 lands on the HEADERS frame where U's stack put it, so the claim is anchored at the record start exactly as before; this only lets a head that U coalesced with such frames be disclosed. It is proven by $\pi_{WIN}$ under $s_{ap,0}$ (anchor $h_2$). N decodes the block with an empty dynamic table, which is sound because the head set `HEADER_TABLE_SIZE = 0` (a block that references a dynamic entry is refused), and requires a `:status` from 400 to 599. A connection closes after an error head is settled.

## 4.8 Settlement

Money follows the same two levels as the protocol: a deposit per connection and a reservation per request. Both go through the node's ordinary billing.

1. **Connection deposit.** At the setup N reserves `connection_deposit_msat`, a value of the signed offer (by default the abandon fee, and at least the session fee), which C accepts only up to a configured maximum. It is **released when the first request of the connection reaches a settlement**: a usage disclosure, an image `accept`, an error head, or a disclosed refusal of the request's stream, that is, any request the provider answered. A connection that ends without a settled request is **charged** the deposit: the client made N garble and send 16 MB and used none of it. The exception is an end that was N's, the relay's or the provider's failure before C could have had a request settled (a 2PC failure on N's side, the provider refusing the handshake or the head, the relay lost); then it is released. The outcome is reported when the connection closes and, when a settlement releases the deposit, in that request's receipt. An idle connection after its first settlement holds no reservation.
2. **Request reservation.** At each request C names the offer it verified by its signature. N keeps an offer it signed for ten minutes after it last served it, and for as long as a connection opened on it is alive, and freezes that offer's per-model rates for the request; an unknown offer is refused and C refetches it. N reserves on the credential (section 4.6) before it accepts any record of the request.
3. **Charge.** N's relay parses R (remove the chunk or frame framing, take the usage event and the terminal marker). The event's model is not a condition: $\pi_{WIN}$ proved that the request names the pinned model, so N prices the usage at the pinned model's frozen rates, and a usage event that spells the model otherwise (an alias, another case) is logged by N's relay and passed to N, not charged as a breach. N charges exactly the usage at the frozen rates, $\lceil (r_{in}(p - c - w) + r_{cached}\, c + r_{write}\, w + r_{out} \cdot o)/1000 \rceil$ msat for $p$ prompt tokens of which $c \leq p$ cached and $w \leq p - c$ written to the provider's prompt cache, and $o$ completion tokens, capped at the reservation, through the node's ordinary finalize path. (If an offer priced cached tokens above $r_{in}$, N uses $r_{in}$ for them; an offer without a cache-write rate bills writes at $r_{in}$. A provider such as Venice prices cache writes above input, so billing them at $r_{in}$ would cost N the difference.) The reservation sizes the prompt at $\max(r_{in}, r_{write})$.
4. **Receipt.** N signs (BIP-340 Schnorr, over canonical JSON) a receipt with the connection id, the request number, model, usage, cost, reservation, the resulting balance, the frozen rates, its key, the SHA-256 of the exact attestation bytes, the released deposit if any, and the time. C requires the signature, takes every field from the signed bytes, and checks the connection id, the request number and the attestation hash against its own records, the usage against the plaintext it already has, and the cost against that usage at the signed rates. Unused balance stays on the credential and is withdrawn through the node's normal refund endpoint.

If C does not disclose after U received the request, N charges the full reservation, which is why disclosure is always in C's interest. The same holds for a response that has no terminal usage event although its status is 200: a stream U cut short, or one that ends in an error event instead of usage. C cannot prove usage that U never reported, and N cannot tell such a response from a withheld disclosure, so it is charged like one (residual risk 11).

**Upstream errors.** For a disclosed error head (section 4.7) the status decides. For statuses the provider answers before doing any work N releases the reservation: on a 429 or 503 (the provider limiting or at capacity) or a 401, 402 or 403 (U refusing N's own key or account, which only the operator can fix) less the session fee $f_s$ if it is still due, on any other 4xx (a request U rejected) less the abandon fee $f_a$; other 5xx are charged like a missing disclosure. In framing "1.1" the head is unambiguous because a strict status line followed by well-formed `Name: value` header lines cannot come from model output or chunk framing; the relay holds it to the start of a TLS record, not provably to the start of the response, so this rests on the provider's response format (residual risk 13). In framing "2" a fabricated head would need a valid HEADERS frame header for the request's stream reachable from byte 0 of a server record through the allowed frames, where the provider's own stack places frame headers; a DATA payload cannot start a record without its own frame header before it, and JSON text has none of the zero bytes every frame header starts with. An error body larger than the disclosure limit cannot be disclosed at all in framing "1.1" and is charged like a missing disclosure; in framing "2" the head is its own frame and the body's size does not matter.

**Refused streams (framing "2").** An HTTP/2 server may refuse a stream without processing it: `RST_STREAM` with `REFUSED_STREAM`, or a GOAWAY whose last-stream-id is below the stream (RFC 9113 §8.7), typically under load or at a graceful shutdown. Nothing ran and U bills nothing, but N cannot see either frame, so without a proof a refusal after the release would be charged the whole reservation, and a GOAWAY before it the deposit. C proves it with

`disclose_reset`: one $\pi_{WIN}$ under $s_{ap,0}$, as for an error head, over the plaintext from byte 0 of a server record no later than the first record of the request's interval to the end of the refusing frame. N walks every frame in it: only stream-0 control frames, WINDOW_UPDATEs, the probe stream's frames and a GOAWAY that still covers the stream may come before the refusal, so a response on the stream (a head, a byte of DATA) stops the walk and no refusal after it can be disclosed. The reset must name the request's stream and code 7; the GOAWAY must carry `NO_ERROR` (a GOAWAY with an error code may be the client's doing). C cannot change the ciphertext N received from U, and it chooses only the record the walk starts at, which N bounds by the interval's start. The request is settled like a 429, $f_s$ if still due and nothing else, which releases the deposit; the connection closes and C runs the request again on a new connection. If U closes right after its GOAWAY, N keeps the request open briefly for this disclosure alone.

**Fees.** N can publish, in the same signed offer, a rate surcharge (already folded into the signed price list), a session fee $f_s$ and an abandon fee $f_a \geq f_s$. C accepts both fees by echoing them in each request; N refuses a request that does not, so a client that does not know about fees is never charged one. The **session fee** pays for the two-party handshake and is charged **once per connection**, with the settlement of request 1; later requests on the connection pay usage only. The **abandon fee** is per request: it is kept when a reserved request is dropped before its body was released. N reserves the amount of section 4.6 plus the larger of the fees that can still fall due. A completed request is charged the usage at the signed rates, plus $f_s$ for request 1, capped at the reservation, and C's receipt check allows exactly that. Both fees are 0 by default.

**Idle and lifetime.** N closes a connection with no request in flight `idle_timeout_s` (a value of the signed offer) after the head was written or after the last settlement, and at `max_requests` or `max_lifetime_s`. The client's pool uses a shorter idle time than the node's, so it normally closes first.

## 4.9 Aborts

| Situation | Outcome |
|---|---|
| Any 2PC check fails, the ClientHello is rejected, or the handshake does not verify | connection discarded before anything is written under $K_0$; deposit charged if the failure was C's, released if it was N's, the relay's or U's |
| A connection is armed or opened and never carries a settled request | deposit charged |
| A request is refused at its `request` message | nothing reserved; the connection stays ready |
| Abort of a reserved request before its body is released (including a failed $\pi_{WIN}$) | U received no complete request; reservation released (less $f_a$); the connection ends |
| U sent response bytes and closed the connection before the body was released (a refusal) | reservation released (less $f_s$ if still due) |
| U refused the stream unprocessed (`REFUSED_STREAM`, or GOAWAY below it; disclosed, before or after the release) | reservation released (less $f_s$ if still due); the deposit released; the connection ends, C retries elsewhere |
| Usage disclosed (chat) or `accept` (images) | charged per section 4.8; in framing "2" the connection is ready again |
| U returned a 429, 503, 401, 402 or 403 (head disclosed) | reservation released (less $f_s$ if still due); the connection ends |
| U returned another 4xx (head disclosed) | reservation released (less $f_a$); the connection ends |
| Released, no disclosure (or another 5xx) | full reservation charged; C keeps its transcript and attestation |

After the release U may already be running the request, so the clients never retry a node error that comes after it on another node or another framing (that could run and bill the request twice), and a caller that cancels after the release does not cut the request: the client lets it finish in the background and settle on the reported usage instead of leaving N to charge the reservation. A connection that ends in the middle of request $k$ does not touch the settlements of requests $1 \ldots k-1$: each has its own attestation and receipt.

### 4.10 Image generation

A request can be an image request instead of a chat completion. The connection, the epochs and the statements are unchanged; what changes is the HTTP request, the price, and how the request settles.

**Offer.** N's signed offer adds `capabilities: ["chat","images"]` and an `images` block: the head template of the image route, the key under which the count is sent, the fixed pins (the provider's paid extras switched off), the parameter space, a maximum count (4), and per image model a list of variants, each a set of pins (a resolution tier and the output format, WebP) with a signed price per image $m_v$ and, optionally, a smaller maximum count of its own. N builds the list from its per-image price books, so only models whose price is a pure function of the pinned parameters are offered (this assumes the provider, too, prices by those parameters alone, whatever else the private part of the body asks for, e.g. an output size within the pinned tier); token-metered image models and providers that report their own cost are left out. The pins are the request's last word, so C does not take them on trust: a fixed pin must be `false` or `[]` (an extra switched off), a variant may pin only a price axis (resolution, quality, format) to a value of the offer's parameter space, and a private option that names a pinned key is refused. A pin that switched a provider feature on (a web search, the prompt written into the image's metadata) would otherwise act on C's prompt. Variants at 4K carry a count of 1: one busy 4K WebP from the case-study provider took 44 s and 5.9 MB of response (a 4.4 MB image; author measurement, raw log not archived), so four in one request could run past the provider CDN's idle limit of about 100 s and turn a paid request into a 5xx after release; the relay's 48 MiB response cap fits four.

**Suffix.** The suffix pins the model, the count $k$, the fixed pins and the variant's pins, in that order and byte-exactly, rendered by N, the relay and C from the signed offer. C's prompt and every option that does not change the price (aspect ratio, seed, negative prompt, safety mode) stay in the private prefix. $\pi_{WIN}$ proves the image suffix exactly as it proves the chat one, so the provider receives no complete request until model, count and tier are proven. In framing "1.1" $\pi_{NK}$ uses the image route's head template; in framing "2" the request's header block names the image path.

**Settlement without a usage proof.** The charge is known when the request is made: $k \cdot m_v + f_s$ (with $f_s$ only on a connection's first request, section 4.8). N reserves $k \cdot m_v + \max(f_s, f_a)$. After the attestation, C sends `accept` for a 2xx response; no disclosure and no $\pi_{C3S}$ run, and N charges exactly the flat amount. For an HTTP error C discloses the response head as in section 4.8 and its error rule applies (the error receipt carries the image terms, and its charge is at most the reservation). A client that sends neither is charged the reservation, which exceeds the flat charge only by $\max(f_s, f_a) - f_s$, so withholding gains nothing. `accept` itself proves nothing: the relay cannot read the response, so it cannot tell a 2xx from a 5xx or from no answer at all, and on `accept` N charges the flat amount whatever happened. An honest C accepts only a 2xx. A dishonest C that accepts after a 5xx pays the flat charge instead of the reservation and so saves at most $\max(f_s, f_a) - f_s$; it gets no image for it. N gains nothing by refusing to charge, and C knows the charge in advance; a refund is the only outcome that needs a proof. The receipt adds the kind, the variant, $k$ and $m_v$, and C checks $cost = \lceil k \cdot m_v + f_s \rceil$ exactly (without $f_s$ after the connection's first request). The attestation is unchanged.

**What N learns.** The model, the variant (tier and format) and the count, as it learns the model and token cap of a chat request; the body length; the response size and timing, which for an image is a weak signal about its content (a flat image compresses better); and success or the error status. It does not learn the prompt, the private options, the image, or, on success, the response head (whose headers flag a blurred or policy-violating image).

**Residuals.** A 2xx with fewer images than $k$ or with an error object in the body is charged in full; C still accepts it (not accepting would cost the reservation) and then reports the shortfall it can see (the framing, the number of images, and the model and count the provider echoes). A 5xx after the body was released is charged the reservation when C discloses the error head or sends nothing (the chat rule, kept until it is known whether the provider bills failed generations), and the flat amount when a dishonest C accepts it, as above. In framing "2" an image request runs on an open connection like any other request (section 6.3); in framing "1.1" it pays a handshake. The response of an image request in framing "2" is one JSON body in DATA frames, and C sends `accept` after the stream has ended.

## 5. Security analysis

**P1 (credential).** The API key exists in the protocol only as plaintext of N's epoch-0 record, encrypted under $K_0$. C never obtains $K_0$, $IV_0$, or any secret from which they can be derived (Z, HS, MS, $c_{ap,0}$, or any outer state $OS_k$): the

2PC outputs these to nobody or to N alone, and nothing is opened afterwards. The secrets C does receive ($c_{hs}$, $s_{hs}$, $s_{ap,0}$, $c_{ap,1}$) are one-way outputs that are not ancestors of $K_0$, the public inner states $IS_k$ give nothing without the outer states (section 4.4), and $h_k$ is a hash of a 28-byte uniformly random value. So a C that records every byte N sends to U, alone or with an observer on N's network path, holds AES-GCM ciphertext under a key it does not know. A cheating C learns nothing from the 2PC beyond its own outputs and the public states, and the API key is not a 2PC input at all. C cannot cut N out of the key exchange: N relays only a ClientHello whose single key share is the joint point. The commitment to $C_1$ that N sends is also safe: without $K_0$ it does not allow testing guesses of the key.

P1 still assumes that U does not send the key back to C (profile item 8), and that N's own process and U are not compromised.

**P1 on HTTP/2: the stored credential cannot be read back.** In framing "2" the credential also lives in the provider's HPACK dynamic table for the life of the connection, and C, who writes every later request, refers to it. We argue that a reference is all C has.

- *An entry is a (name, value) pair.* HPACK has two ways to use an entry. The indexed form emits the whole pair, here `authorization: Bearer <KEY>`, which is what an authenticated request needs. The name-reference form reuses only the entry's name with a new literal value. No encoding reuses a stored value under another name, so C cannot move the key into a header that the provider echoes (a user agent, a trace id).
- *Tables are per direction.* The entry is in the table that decodes C-to-U header blocks. U's response header blocks are encoded against a separate table, which the head set to size 0. A response cannot point into the request table.
- *No partial-match oracle.* HPACK compresses by whole-field match, and the compressor for this direction would be C's own, which never holds the value. There is no shared compression context between attacker-chosen text and the secret, the condition of CRIME-style attacks: C cannot ask "is the entry equal to X" through the size of anything, because nothing that is sent is compressed against the entry by a party that knows it.
- *No push.* The head disables server push, so U sends no synthesized request header blocks.
- *C's header blocks are pinned in full.* $\pi_{WIN}$ proves every HEADERS record equal to a public block, so C cannot choose a method or path that reflects request headers (`TRACE`, a debug endpoint), and cannot add an entry or resize the table.

What remains is the provider's application copying the request header into a response, which is the same condition as on HTTP/1.1 (profile item 8) and is checked per provider (Appendix A). The dynamic-table entry is scoped to the connection: it does not survive it and is not visible on any other connection.

**P2 (content).** The requests run under $c_{ap,1}$ and the responses under $s_{ap,0}$, which the 2PC outputs only to C. N's outputs ($c_{hs}$, $s_{hs}$, $K_0$, $IV_0$, the anchors, the inner states) are not ancestors of either. A cheating N that garbles a wrong circuit, for example one that outputs C's keys in a form N can read, is caught by C's consistency check before C uses any key, except with probability about $2^{-60}$ per connection (below), and learns at most 3 bits about C's fresh, single-use inputs and the connection's internal wires (with ECtF's semi-honest conversion as a stated assumption, section 4.4). The proofs that decrypt under C's keys ($\pi_{WIN}$, $\pi_{C3S}$) output one equality bit, on parameters C has checked against its own records, so N cannot use them as a decryption oracle. $\pi_{WIN}$ needs care here because its windows are chosen per record: C proves only windows that the frame rules make public (frame headers, content-type bytes, the header block, the suffix), and refuses a window that reaches into a DATA payload before the suffix. What N learns in plaintext is public by design: the request's header block, the frame lengths, the suffix, the usage event, or on an error the response head.

**Soundness of the 2PC against a cheating N.** N fixes every base mask and every product $t_k$ under $\Delta_B$ before it sees the challenges, and learns nothing about $\Delta_B$ (security of the correlated OT). The product check fails for a wrong $t_k$ except with probability $2n/2^{128}$ for $n$ gates. If some AND gate of a stage was evaluated inconsistently, then $E = \sum_k \chi^k e_k \neq 0$ except with probability $n/2^{128}$ over $\chi$, and C then accepts only if the 64-bit projection of $\Delta_B E$ equals a value N fixed without any information about $\Delta_B$. For $E \neq 0$ the map $\Delta \mapsto \pi(\Delta E)$ is onto $GF(2)^{64}$, so with 127 uniform bits in $\Delta_B$ this happens with probability at most $2^{-63}$ per stage, at most about $2^{-60.6}$ for the five stages. N cannot grind: it learns nothing about $\Delta_B$, and the first failure ends the connection. Once every gate of a stage is consistent, C's outputs of that stage are the correct function of its own inputs, N's effective inputs and the real digests, because C's inputs and the digest wires are pinned by the pointer checks and C decodes with authenticated masks. The digests themselves are public values that C computes too, from $IS_k$ and messages it knows, so N cannot feed a stage a digest of another message.

*The salt.* Truncating the rows to 64 bits needs one more step. With $E = 0$, a label that is not N's honest label but has the right pointer bit changes no output; it only changes the check sum by the difference of two truncated hashes. Without a salt, a node that reused its labels across connections could precompute once, with about $2^{64}$ work, a table deviation whose 64-bit hash difference is zero. That deviation would pass the check and could tell N which of two evaluation paths C took: leakage without an abort. Because the row hash is keyed by a salt from C's seed, and everything that fixes such a label (tables, $\Delta_A$, N's keys) is committed before the salt is known, the hash difference is a fresh 64-bit value and the check fails except with probability $2^{-64}$. Labels N chooses online (its own inputs, the digest labels, the corrections for C's inputs) are chosen when N knows the salt, but a digest label for the other bit value has the wrong pointer bit, and other shifts are by amounts N already knows, so they carry no information about C.

*Label-check pairs.* When C reports an output label to N, the report itself must not become a channel. A bare hash of the reported labels would let N test candidates offline. Instead N commits in preprocessing to the salted hashes of both labels of every wire C will report, and C reveals a label only if it matches one of the two. A report therefore tells N one bit per wire that N is entitled to (the output), and a mismatch is an abort like any other of the stage.

**P3 (integrity).** C's records are authenticated under a key only C has. N holds $K_0$ and can write any epoch-0 records (below), but none of them carries C's data. The suffix is proven to be the body's tail, so the effective parameters are exactly the agreed ones; response tags are verified by C. In framing "1.1", N cannot see inner content types, so C could hide a KeyUpdate of its own among the body records; U's view of the final record is then the suffix XORed with two keystreams C cannot choose, which is not valid JSON, so U rejects the request, and the relay's length accounting makes padding and non-data records equally useless. In framing "2" the question does not arise: every record's content type and frame header are proven, so a hidden KeyUpdate, alert or padded record fails $\pi_{WIN}$ before it is written.

**The frame rules, against a cheating C.** In framing "2" the claim is that U's HTTP/2 stack sees, for each request, exactly one HEADERS frame equal to the public block and DATA frames on the same stream that end with the suffix, plus control frames that carry no request. The argument has three steps. (1) Record boundaries are N's: N relays whole records. (2) For each record, the proven frame header states a payload length of the inner length minus 10, and the proven last byte is the content type, so the record's plaintext is exactly one frame and no padding; U's parser, which is aligned at the start of C's first record (N wrote everything before it), stays aligned at every record boundary. (3) The proven header fixes the frame's type, flags and stream. So C cannot place a frame header anywhere but at byte 0 of a record, where it is proven. The gates include both evasions we know: a HEADERS frame hidden in a DATA payload, and TLS padding that shifts the boundary (section 6.5). The DATA payload before the suffix is C's private content, as the body prefix is on HTTP/1.1, and the provider's JSON parser resolves it against the suffix by the same profile items 5 and 6. A control record's header is proven against its declared kind and its payload length is fixed (8 bytes for a PING ACK, 4 for a RST_STREAM, none for a SETTINGS ACK), so it cannot hold a frame; C sends no WINDOW_UPDATE at all, whose increment would be a payload of C's choosing that makes the provider close the connection.

**Cancelling a request does not free its credential reference.** Suppose C force-stops request $k$ with RST_STREAM and starts another HTTP request that names the stored credential, hoping that U serves it while N bills nothing. N accepts a record as part of a request only inside an accepted `request` (which it reserved), and classifies and proves every record before writing it. So (a) a HEADERS record with no `request`, for the next stream or the same one, is refused before it is queued and never reaches U; (b) RST_STREAM is accepted only after all of the request's records are in and queues behind them, so it cannot overtake an unreleased last DATA record, and U holds a body without END_STREAM, on which it does no inference (profile item 1); the request then ends before its release and costs the abandon fee; (c) after the release a request settles only on its own usage event or a provider error head, and otherwise its full reservation is charged, so a cancelled request never costs less than its bound; (d) a control record cannot carry a frame (above); (e) in framing "1.1" the head pins `Content-Length` and `Connection: close`, content beyond the body length is refused, a second request inside the body is body bytes, and TLS padding only shortens what U receives, stalling C's own request; (f) the HPACK table belongs to one HTTP/2 connection, so a new TCP/TLS connection has no index 62, and C never holds $K_0$ to write a credential itself. Nor can request $k$ hide in request $k+1$: N accepts $k+1$ only after $k$ settled, and only a usage settlement keeps the connection, so $k$'s generation is over by then; stray frames of stream $k$ in $k+1$'s interval make $k+1$'s tail grammar fail (it accepts only streams 0, 1 and $k+1$'s), which costs C, not N; and a usage event names its stream in the frame headers N parses. The node gate runs each form with exact money (section 6.5).

**P4 (accounting).** The billed usage is a substring of the authenticated response, proven under the connection's server secret and anchored to the end of the request's interval of server records. The proofs check AES-CTR decryption, not the GCM tag: N takes the ciphertexts from its own transcript, never from C, and a tag check would add nothing, since C holds $s_{ap,0}$ and could compute server tags itself (residual risk 9). The anchors $h_1$, $h_2$ are outputs of the same 2PC that produced the keys, and N accepts them only through C's label-checked output report, so C cannot prove against a secret of its choosing without a SHA-256 preimage. N computes the suffix from the request's values, accepts only that suffix, and lets U see the complete request only after $\pi_{WIN}$ has verified, so U runs exactly the reserved model and cap. On a connection with several requests, the interval rule keeps a usage event to its own request: a cheaper event from an earlier request lies before $S_k$, and a later one cannot exist because requests are sequential and the stream must end inside R. The receipt binds the result to the connection id, the request number and the reservation.

**P5 (authenticity).** Both parties verify the certificate chain, hostname, CertificateVerify and both Finished messages: C in its TLS stack, N from the handshake secrets that the 2PC gives it in the clear. In framing "2" both also check that U selected ALPN `h2`.

**P6 (freshness).** The connection id is a hash of fresh nonces and the node's one-time reservation id; the TLS randoms, the ECDHE scalars, the garbled tables, both global offsets, the MAC keys and the salt are fresh per connection. Within a connection, requests are numbered, each has its own reservation, stream id, interval, attestation and receipt, and N accepts request $n$ only after $n-1$ has ended.

**Residual risks.**

1. *N controls epoch 0.* N holds $K_0$, so it can write any number of records in epoch 0, not only the proven head, and $\pi_{NK}$ binds the head's content, not the bytes N writes. N could, for example, use another account or endpoint, or add a header that enables provider-side retention. Such a head can also change how U processes C's body wherever the provider offers a header or endpoint that does so: route it to another backend, add a system prompt, change moderation or caching (profile item 12). C cannot see N's wire bytes and cannot detect this. The effect is bounded: epoch 0 carries none of C's data, C's own records stay authenticated under $c_{ap,1}$, the suffix still sets the parameters that are in the body, the response is readable only by C, and a broken request shows up as a failed response. A node could always choose to misuse its own account; what the protocol guarantees is that it cannot read or alter C's content in transit; how U treats that content rests on profile items 10 and 12. Whether the provider keeps a copy that the account owner can read, or forwards content on its behalf, is profile item 10, not something the protocol can rule out. In framing "2" the epoch-0 record also sets the connection's SETTINGS; a dishonest N could choose others, with the same bound.
2. *The 2PC is a new, unpublished variant.* The garbler-only authenticated garbling, its staged consistency check with garbler-held masks, the 64-bit projection, the salt, the label-check pairs and the split of HMAC with public inner states are argued in section 4.4 and above, but this construction has no published proof and has had no external review. The security of the public states rests on two compression-function assumptions (section 4.4) instead of on HMAC as a PRF. The three-halves garbler is our own implementation: it was rebuilt from the scheme's structure, with the evaluator's matrices taken from a public reimplementation, and it is checked exhaustively for correctness. Correctness does not depend on those matrices but the privacy argument does, so they must be checked against Rosulek and Roy's paper. The MAC layer assumes that `mpz2`'s Ferret and KOS are secure against a malicious receiver (here N). ECtF is semi-honest, as in TLSNotary, and the leakage bound of risk 3 assumes that it gives a malicious N nothing beyond its outcome.
3. *Selective failure.* Each connection can leak at most 3 bits through the abort pattern: N chooses the predicate, which can be any function of the client's private inputs and of the circuit's internal wires, including $s_{ap,0}$ and $c_{ap,1}$. The bound assumes ECtF leaks nothing beyond its outcome (risk 2). No 2PC input, garbling key or traffic secret is reused across connections, so the bits never accumulate on any secret, and the API key is not a 2PC input. The bound is per connection, and a connection now serves many requests; v2 leaked up to 2.8 bits per request. A node that aborts connections on purpose is visible to its clients.
4. *P-256.* The key exchange uses P-256, as most TLS clients still offer; X25519 cannot be used with an additive joint share.
5. *Traffic analysis.* N sees the size of every record. The body length is visible, and so is the response's token-length sequence, because streaming providers emit one record per token or small token group. This side channel has been shown to reconstruct a large share of LLM responses (Weiss et al., USENIX Security 2024). Compared with a node that sees the plaintext it is strictly less exposure; compared with "ciphertext only", it is more than

volume and timing. The prompt side can be padded inside the JSON body; the response side needs provider-side padding or batching.

6. *DoS and limits.* The node and its relay validate the setup and reserve the deposit before any 2PC work, and reserve each request before accepting its records. They bound concurrent connections (32; a connection takes one slot for its life), the time to setup, the setup frame, the 2PC (180 s), the body (8 MiB), the requests per connection, its lifetime and its idle time. A client address may hold only a bounded number of connections that have not yet released a body (twelve by default) and of connections in all (sixteen); when the node or an address is full, a new setup first closes the longest-idle connection that already settled a request (it holds no deposit) and is refused only when there is none, so cheap, mostly idle connections cannot hold every slot. At most eight requests in a row may be refused on a connection. Connections that have not reserved yet are a separate, smaller pool in which the oldest is evicted first. The relay's withdrawal of framing "2" counts only failures no client can cause (no answer to the canonical ClientHello, `h2` not selected, a close right after the connection head with no client record written), so no client can switch it off for others. Opening a connection costs N about 16 MB of upload and 0.4 s of CPU, which the deposit prices (section 8.1).
7. *Error metadata.* On an upstream error N learns the status line, the response headers and the body length, but not the body.
8. *Endpoint hygiene.* Keys live in C's process; N holds the API key in memory. C's certificate validation is RFC 5280 path validation to the WebPKI roots without revocation checks.
9. *Interactivity.* There is no artifact a third party can check. C holds $s_{ap,0}$ and could forge server records, so a portable artifact would need a design in which no single party holds the server key.
10. *Provider drift.* A provider change that breaks parameter pinning or usage reporting turns a verified session into an unverifiable one, which clients treat as failure, not fallback. The profile gate must be re-run per provider.
11. *Balances after an abort.* If N cuts the stream before the usage event, it charges the reservation, and C cannot prove who aborted. This is the trust the existing Routstr design already places in the node; in the honest case the cost is computed from upstream-authenticated usage, and the receipt binds all of it. The amount at N's discretion is the reservation for the requested model, per request, plus the deposit of a connection that N ends before its first settlement.
12. *The relay trusts its peer.* N's relay holds the API key but does not check client credentials: N validates the credential and reserves before it connects to the relay. Whoever can reach the relay directly therefore spends the provider key without paying. The relay requires a bearer token shared with N unless it listens on loopback; on loopback without a token every local process on that host can use it. The token travels over plain HTTP, so off the host the N-to-relay link needs a private network or an encrypted tunnel.
13. *Error heads rest on the provider's format (framing "1.1").* An error disclosure is held to the start of a TLS record, not provably to the start of the response (section 4.8). With the case-study provider's JSON bodies, whose client-controlled text is always escaped, no header-like block can be formed later in a response; a provider whose bodies carried raw CRLF lines next to client-controlled text would let C present a fabricated error head and get a refund. In framing "2" the equivalent forgery needs a binary HEADERS frame header for the request's stream at byte 0 of a server record in the request's interval, which is a smaller surface, but it still rests on how the provider's stack maps frames to records.
14. *Provider headers.* C reads the whole response, including headers in which the provider reports the account's balance and rate-limit counters. Every client therefore learns the operator's provider balance and, by sampling the counters across its own sessions, roughly how many requests other clients sent through the same key. The protocol cannot hide them from C; the clients drop them before handing headers to an application.
15. *Proof-channel admission.* N admits a socket to a live connection's proof and 2PC channels by its sid and a random token N gives only C (in `ready`; unlike the sid it is in no receipt or attestation, and N's logs drop it). Whoever learns both from C could take a channel before C does and so end the connection; after a body is released that charges the reservation, as an abort by N would. Such a party cannot read C's channel (each channel has one peer) or prove anything for C.
16. *Linkability on a kept connection.* Requests on one HTTP/2 connection are linkable for the provider: same TCP connection, same TLS session. Requests under one API key are already attributed to one account, but a provider (or its CDN) that would not otherwise group requests from different users of a node can group those that share a connection. The client's pool therefore never shares a connection between two *reuse scopes*; the daemon derives the scope from its local client by default, so all requests made with one local API key are connection-linkable, across conversations and applications sharing it. A request header narrows the scope (one conversation); a caller

that needs unlinkable requests uses `reuse: none` or framing "1.1". A passive observer of the node's link to U sees one long-lived flow per connection, and a global observer can correlate the client's and the node's traffic by timing and size. The same holds towards N: a client that pays each request with a fresh e-cash token is linked across them by the connection. A kept connection is also longer-lived state: one pair of traffic secrets protects all its requests (C may not KeyUpdate under the frame rules), so a live compromise of C's process exposes the connection's current and later traffic, and secrets retained after it ends would expose the whole recorded connection. Clients clear the secrets they hold at the close and keep them out of logs, but that is best effort, not cryptographic erasure: the clients are JavaScript, and copies made by the runtime (garbage collection, heap snapshots, swap, core dumps) may remain. The bound is the connection's scope: operators keep the request and lifetime limits in proportion, and a user who needs a secret pair per request uses `reuse: none` or framing "1.1"; a per-request ratchet needs a new anchor protocol. Linkability is by design; the scope, `reuse: none` and framing "1.1" are the tools to limit or avoid it.

17. $\pi_{WIN}$*, the frame rules and the interval rule are new.* The argument above that the rules pin U's view of a request, and that the interval rule pins a usage event to its request, is ours and has had no external review. HTTP/2 has more parser state than HTTP/1.1 (CONTINUATION, padding flags, priority, stream states), and an implementation difference between N's expectations and U's stack is where we would look for a break.
18. *Provider drift on HTTP/2.* Framing "2" needs the provider to keep the dynamic-table entry, accept a KeyUpdate after the first flight, and honour a response table of size 0 (section 3.1). A provider that changes one of these breaks the framing, not the guarantees: requests fail, the relay stops offering "2" after five failures, and clients fall back to "1.1".
19. *The HPACK argument rests on the specification.* The read-back argument (above) holds for a conforming HPACK decoder. A provider stack with a non-standard extension that exposes table contents, or an intermediary that re-encodes requests and logs them, is outside it, as a header-echoing endpoint is outside the HTTP/1.1 argument. The credential stays in the provider's (or CDN's) decoder state for the connection's life: v3 does not reduce provider-side exposure.
20. *A malicious N can change the request table.* Since $\pi_{NK}$ binds only the head (risk 1), N can write further HEADERS frames in epoch 0 that add or evict table entries, so that index 62 no longer names the proven credential entry, or that change the connection's state. C's own header blocks are still proven, and this reveals neither the prompt nor the key, but it can change how C's requests are authenticated or make them fail.
21. *Receipts and attestations are linkable.* The per-request attestations and receipts of a connection repeat its id, transcript hashes, $c_{commit}$, the certificate-chain hash and the request numbers and intervals: anyone shown several can link them and see their order and gaps. They are private documents of the client; unlinkable presentation to third parties would need a new signed format (redacting fields of the signed ones breaks their signatures).
22. *Body size and error heads.* The body length is exact unless C pads; the clients can pad with JSON whitespace to a bucket (off by default, since N learns the prompt's token count at settlement anyway). An error head is disclosed from byte 0 of its record, with only stream-0 control frames, WINDOW_UPDATEs and frames of the probe stream allowed before it; a provider that splits a frame across records right before an error head makes that error undisclosable, and after the release the request is then charged in full (profile item H9). Late answers to the probe stream are skipped by the usage-tail grammar, so billing does not depend on when they arrive.
23. *Refused streams.* A disclosed refusal (section 4.8) rests on the provider's frame boundaries as an error head does, and needs the records from the request's interval to the refusal to be walkable (at most 8 records and 16 KiB, no KeyUpdate); otherwise the request is charged by what N saw, as before. A client can still end an accepted request it never wrote for $f_s$ by holding it until the provider's idle timer sends GOAWAY (at least `h2.idle_secs`): the provider did no work, and the gain is at most $f_a$, or the deposit less $f_s$, per connection.

# 6. Implementation and evaluation

## 6.1 Prototype

The implementation consists of:

- `cu-crypto`: native TLS 1.3 key schedule and GCM, tested against RFC 8448 and NIST vectors.
- `twopc`: the two-party key schedule (ECtF over P-256, the split circuit and its five stages, a three-halves garbler and evaluator, IT-MACs on N's masks from `mpz2`'s Ferret and KOS correlated OT, the product and consistency

checks, the label-check pairs), with a client-side binary.

- `zkbench`: the VOLE-ZK statements $\pi_{NK}$, $\pi_{WIN}$ and $\pi_{C3S}$ with tests that the circuits agree with native code, mutation tests, and one-bit disclosure tests; the client's prover binary reads its job and witness on stdin, so no secret touches the disk.
- `cu-sidecar`: N's relay (Rust). It enforces the ClientHello and ServerHello rules, runs N's side of the 2PC, verifies the handshake fail-closed, assembles and writes the epoch-0 records, holds C's records until their proof, parses HTTP/2 frames and HPACK on the response side as far as the disclosures need, and verifies C's proofs.
- A reference TypeScript client, and the production path: a transport in the Routstr client SDK with a connection pool, used by the client daemon, and a WebSocket route in the node that relays to the sidecar and bills through the node's ordinary accounts.

At the time of writing the HTTP/2 path is a prototype that runs end to end through the node, the SDK and the daemon against the live provider; the browser library does not have it yet.

C's TLS stack is a fork of an MIT-licensed TLS 1.3 client with two hooks: an external key share, and traffic secrets injected from the 2PC instead of computed locally. It accepts only the version, suite and group it offered, requires the server's flight in RFC 8446 order with each message once and a CertificateVerify over the validated chain before it accepts the Finished, validates certificate paths per RFC 5280 and hostnames per RFC 6125 against the same Mozilla root set as N's relay (generated from the relay's pinned copy), and uses the full 64-bit sequence number in the nonce.

## 6.2 Component costs

All component figures are from an Apple M2 Max, release builds (2PC: `crypto/twopc/BENCH.md`, 2026-11-11; $\pi_{NK}$ and $\pi_{C3S}$: `crypto/zkbench/BENCH-v2.md`, 2026-10-10, unchanged statements). "Local" has no delay. Network figures use an emulated link with a rate per direction and a fixed one-way delay. The v2 columns are v2's own measurements on the same machine. Memory was measured on macOS only.

**Two-party key schedule.**

| | v2 (garbled in full) | v3 (`split-public-1`) |
|---|---|---|
| Garbled AND gates | 656,179 | 479,022 |
| Garbled SHA-256 compressions | 31 | 22 |
| C receives | 22.12 MB | 16.38 MB |
| C sends | 1.21 MB | 1.13 MB |
| Cold run, local | 0.67 s | 0.45 s |
| Online round trips: handshake / application | 1 / 1 | 3 / 1.5 |
| CPU per run: C / N | 0.44 s / 0.53 s | 0.31 s / 0.40 s |
| Peak memory, one run: C / N | 410 MiB / 517 MiB | 353 MiB / 457 MiB |
| Peak memory per additional concurrent run, N | 300 MiB | 240 MiB |

Of the 16.4 MB C downloads, 11.8 MB are the three-halves tables and 3.9 MB the 64-bit check rows; the mask commitments, the label-check pairs (81 kB) and the correlated OT are the rest. Of the 1.13 MB it uploads, 1.09 MB is its part of Ferret, in which C is the sender. Locally the handshake stages take 37 ms and the application stages 29 ms at C. The split adds round trips on the request path: at 80 ms round-trip time the handshake stages take 0.32 s and the application stages 0.22 s, against 0.13 s and 0.15 s for v2's two phases. On a connection that carries many requests this is paid once.

**Proofs.**

| Proof | Local | Data | SHA-256 compr. | AES blocks |
|---|---|---|---|---|
| $\pi_{NK}$ (connection head; benchmark: a 228-byte HTTP/1.1 head) | 0.16 s | 1.45 MB | 5 | 15 |

| Proof | Local | Data | SHA-256 compr. | AES blocks |
|---|---|---|---|---|
| $\pi_{C3S}$ (usage; one 596-byte server record) | 0.20 s | 1.47 MB | 7 | 38 |
| $\pi_{WIN}$ (a request's windows), over a pre-warmed proof channel | 39-66 ms | about 610 kB down per request together with $\pi_{C3S}$, moved off the request path by the top-up | 7 | 1 per 16-byte block touched |

The first two rows include the proof's own OT setup; over a pre-warmed proof channel $\pi_{NK}$ took 0.06 s and $\pi_{C3S}$ 0.04 s in v2's live runs. $\pi_{WIN}$ needs about 160 k correlations plus 6.4 k per AES block: 308 k for a small chat body and 457 k for a 200 kB body in 13 DATA records, whose windows touch more blocks. $\pi_{C3S}$ adds 4 compressions per server KeyUpdate it steps through (at most 16). $\pi_{NK}$ for the HTTP/2 head and $\pi_{WIN}$ with per-proof setup have not been benchmarked separately: `[TO MEASURE]`.

## 6.3 End to end

**Final build, through the node.** Connections against api.venice.ai through a node on loopback (reference client → node → relay → provider, `mistral-small-3-2-24b-instruct`, `max_tokens` 12; medians; time to first token counted from the connection's start for a cold request, from the claim for an armed connection, from the request on an open one; author measurements, 2026-11-12):

| | Framing "2" | Framing "1.1" |
|---|---|---|
| Cold request (connection opened for it): time to first token | 1.38 s | 1.52 s |
| Request on an armed connection | 0.81 s | 0.76 s |
| Request on an open connection | 0.57 s | n/a (one request per connection) |
| Cold: C sends / receives | 2.63 MB / 17.97 MB | about the same |
| Request on an open connection: C sends / receives | 86 kB / 12-15 kB (plus a 610 kB top-up between requests) | n/a |

Of the cold 17.97 MB, 16.38 MB are the 2PC and about 1.6 MB the proof channel's correlations; of the 2.63 MB sent, 1.13 MB are the 2PC and 1.5 MB the proof channel. A request on an open connection would need about 560 k fresh correlations for $\pi_{WIN}$ and $\pi_{C3S}$. Without a top-up the channel extends online during the request (0.5-1.45 MB down on the request path, measured). With the top-up (§9.2 of the specification) the client asks for one Ferret iteration of 900 k correlations after the connection head and after each settled request, while no request is in flight: about 610 kB and 170 ms natively, off the request path. Measured on five requests 3 s apart on one connection: the request path then carries 12-15 kB down, and the proof channel downloads nothing during requests 2-5. It does not depend on the 2PC. The time to first token is dominated by the provider's own latency in every row. Earlier prototype runs (2026-11-11, before `split-public-1` was merged, so with v2's 22 MB 2PC in the open: 23.7 MB / 2.6 MB per open, 0.5-1.0 MB / 85 kB per later request, the body released about 60 ms after it was sent, time to first token through the daemon 0.46-1.26 s on a reused connection against 0.9-1.5 s for pre-warmed v2) are superseded by these. They do not show that v3 is faster than pre-warmed v2 for the first token: both are within the provider's latency; what v3 removes is the repeated 2PC traffic and the pre-warm prediction.

**What became of pre-warming.** In v2 every request paid a two-party handshake, and pre-warming hid it: the client armed sessions ahead of requests, each bound to a model, a token cap and a body length, each holding a full-context reservation, and it padded bodies to the armed length. In v3 an armed connection is bound only to the credential and holds only the deposit. On framing "2" a kept connection replaces pre-warming for every request after the first; the client may keep one armed connection per pool key while there is traffic and every open connection is busy, so that a burst does not wait for a 2PC. On framing "1.1" armed connections work as v2's armed sessions did, without the

binding, the padding and the reservation. The proof channel is still prepared ahead: it holds the OT/VOLE correlations for $\pi_{NK}$ and the first $\pi_{WIN}$ and $\pi_{C3S}$, and on a kept connection it is topped up between requests, one bounded step at a time, at the client's request.

**Client pool.** The SDK keeps connections per (node, credential, reuse scope). A request takes an idle connection of its key or opens one, up to a limit per key and in total; parallel requests use separate connections, since requests on a connection are sequential. One-off e-cash tokens use framing "1.1" or a connection per token.

**Shaped links.** The same runs with the relay behind a rate-limiting userspace proxy (one shared link per direction, 40 ms one way; reference client to the relay directly, no node), medians of 3:

| Link (up / down for the client) | framing | cold | armed | open |
|---|---|---|---|---|
| 2 / 50 Mbit/s | "2" | 12.1 s | 1.43 s | 0.92 s |
| 2 / 50 Mbit/s | "1.1" | 11.8 s | 1.38 s | n/a |
| 2 / 10 Mbit/s | "2" | 18.9 s | 1.59 s | 1.23 s |
| 2 / 10 Mbit/s | "1.1" | 19.2 s | 1.85 s | n/a |

At 2 Mbit/s up the cold time is the 2.63 MB upload (about 10.5 s): the proof channel's 1.5 MB shares the uplink with the 2PC, so the 2PC-only figure of section 6.4 (7.0 s) is not the time to first token.

**Against a deployed node.** One run on 2026-11-12, right after the deployment, from a home connection to routstr.cypherpunk.today (reference client, `deepseek-v4-flash`, 30 output tokens; one sample, not a benchmark): framing "2", the request that opened the connection reached its first token after 6.0 s (5.0 s of it the connection: 2PC preprocessing 2.9 s, handshake, $\pi_{NK}$), the two later requests on the same connection after 1.3 s and 1.0 s, with 14-15 kB from the node on the request path and one 610 kB top-up between them; framing "1.1", a cold request after 4.7 s. For v2, with the 22 MB download, a cold request from a home connection reached its first token after 6.7 to 21 s and a pre-warmed one after 1.1 to 1.5 s (2026-10-19, `deployed-cold.jsonl`, `deployed-arm.jsonl` in the evidence directory).

## 6.4 Over a network and bandwidth

A cold 2PC run (preprocessing, key share, ECtF and all stages, until both parties finish) on an emulated link, median of 3 runs. The dual-execution column is the first implementation of v2 (section 6.6), single runs on the 2 and 7 Mbit/s uplinks.

| Link (one-way delay; up / down for the client) | Dual execution | v2 | v3 |
|---|---|---|---|
| local | 0.17 s | 0.67 s | 0.45 s |
| 40 ms; 2 / 50 Mbit/s | 85.2 s | 7.42 s | 7.02 s |
| 40 ms; 7 / 50 Mbit/s | 24.9 s | 5.50 s | 4.65 s |
| 25 ms; 100 Mbit/s symmetric | 2.27 s | 3.14 s | 2.68 s |
| 40 ms; 2 / 10 Mbit/s | 85.2 s | 20.12 s | 15.61 s |
| Client sends / receives | 21.1 / 21.1 MB | 1.21 / 22.12 MB | 1.13 / 16.38 MB |

On a 2 Mbit/s uplink with a fast downlink C's Ferret upload is the critical path (1.09 MB, about 4.4 s), so the smaller circuit gains little there and the extra online round trips cost 0.3 s. Where the download is the bottleneck (10 Mbit/s) v3 saves 4.5 s: 16.4 MB take 13.1 s of the 15.6 s.

**Data per request.** v2 moved about 23.3 MB of 2PC traffic and 4.4 MB of proofs for every request, about 28 MB, of which N uploaded about 24 MB. With framing "2" on the final build, opening a connection moves 17.97 MB to the client and 2.63 MB from it, and every later request about 1.0 MB and 86 kB. For a connection of $r$ requests the average per request is about $(18 + 1.0\,(r-1))/r$ MB to the client: 18 MB for one, about 2.7 MB for ten, about 1.2 MB for a

hundred. (With the prototype's v2-size 2PC the open was 23.7 MB; the same arithmetic then started at 24 MB.) The 2PC cost is constant in the size of the prompt and the response; $\pi_{WIN}$ grows with the number of body records (two AES blocks per DATA record), not with their content, and a body of more records than one $\pi_{WIN}$ holds is proven in several batches before any of it is written (section 4).

**What reuse needs.** The large saving holds only for sequential, settled requests that share a node, a credential and a reuse scope: parallel requests need connections of their own, an error head ends the connection, and one-off e-cash bearers, `reuse: none` and framing "1.1" keep one connection per request. Every reused request still costs a reservation, a `request_ok` round trip, $\pi_{WIN}$, $\pi_{C3S}$ (or `accept`) and a settlement; the node keeps the provider socket and the proof channel of an idle connection. At commodity prices the megabyte and two proof verifications of a reused request are worth well under a millisatoshi, so usage at the frozen rates covers them except for the smallest requests; an operator who wants a margin sets the rate surcharge (`PROTOCOL-V3.md` §8.5).

## 6.5 Live verification

**Provider behaviour.** `provider-profile-check.py --h2` checks what framing "2" depends on (section 3.1) and passed against the case-study provider on 2026-11-12 with 300 s of idle and an 8 MiB body: ALPN `h2` with the client's one suite and P-256; a TLS KeyUpdate directly after the connection head accepted; `HEADER_TABLE_SIZE` left at 4096 (the credential entry is 94 bytes); response headers decodable with an empty table; a request naming `authorization` by index authenticated before and after the idle time (and refused without the field); 8 MiB in 1026 DATA frames answered; no reflection of the credential on seven response paths; the event stream ending as the usage disclosure parses it. The check refuses any argument it does not know, so a mistyped mode cannot pass for a check that did not run.

**Honest runs.** Sequential chat requests on one connection, bursts over several connections, image requests on an open connection, armed connections, refused requests followed by good ones, forced framing "1.1" and the fallback from a failing framing "2" completed through the node and the daemon with all proofs verified and signed receipts passing every client check; the gates compare the node's database balance with the expected money outcome of every case.

**Attacks.** The gates (`crypto/tools/v3-check.sh` against the relay, `v3-node-check.sh` through a node, with exact money) run a dishonest reference client with an expected outcome per case. Each of these was refused before the provider received a request it would bill wrongly:

- a HEADERS frame hidden inside the payload of a body record;
- TLS padding used to shift a frame boundary;
- a suffix other than the canonical one, another model's suffix, another cap;
- a header block naming another path;
- a usage event replayed from an earlier request of the same connection;
- a tampered image suffix;
- records before `request_ok`, the client's own key share, a v2 2PC peer;
- a request force-stopped with RST_STREAM so that a later one could ride on the stored credential unbilled (section 5): a HEADERS record without `request`, RST_STREAM before or after the release, a control record carrying a HEADERS frame, and on framing "1.1" a record after the final one or a second request inside the body.

An honest control and a v2-socket control run in the same gates.

**Scope.** One provider was tested. The HTTP/2 behaviours above are properties of that provider's front end on the day of the test.

## 6.6 The dual-execution version

The first implementation of v2 used dual execution (Mohassel and Franklin 2006; Huang, Katz and Evans 2012) instead of node-only garbling. Each phase was garbled twice with `mpz2`'s half-gates garbler: N garbled and C evaluated, and C garbled and N evaluated, both runs at once. Every output was decoded by both evaluators, and before using any output the parties compared the two runs through a commit-then-open hash of output labels, which a party can produce for a differing bit only with an inactive label of the other's circuit. Nothing was ever opened. Per phase the adversary

learned the outcome of the comparison, $\log_2 3 \approx 1.6$ bits per session about fresh values, under the same invariant as now.

It worked and was deployed, but each party had to garble 656,179 AND gates and send the other about 21 MB of tables. The client's upload made a cold session take 25 s on a 7 Mbit/s uplink and 85 s on a 2 Mbit/s one (section 6.4), about 35 s to the first token from a home connection, and a minute of background upload for every pre-warmed session. Node-only garbling removes that upload. Dual execution remains a sound alternative: it is simpler, it rests on better-studied constructions (half-gates garbling and the dual-execution equality check), it leaks less (about 1.6 bits per session against 2.8 for v2's node-only garbling and 3 per connection now), it is faster on fast symmetric links, and it needs no Ferret. It is kept as the tag `v2-symmetric` in the three repositories (routstr-confidential `0876b5e`, routstr-core `dd9e51a5`, routstr-sdk `b5d16f0`).

# 7. Earlier versions

## 7.1 From v2 to v3

Protocol v2, the previous version of this paper, had the same roles, the same split of secrets and the same guarantees. Its unit was a *session*: one two-party handshake, one HTTP/1.1 request, one response, and the provider closing the connection. Every request therefore cost 22.1 MB from the node to the client and 1.2 MB back, and about 7 s of 2PC on a home link with a 2 Mbit/s uplink. v2 hid this with pre-warming: sessions armed ahead of requests, each bound to a model, a token cap and a body length and each holding a full-context reservation.

Two things could be reduced: how often the handshake is paid, and what it costs.

| | v2 | v3 |
|---|---|---|
| Unit | a session, one request | a connection that carries requests $(sid, n)$ |
| HTTP to the provider | HTTP/1.1 | framing "2" (many sequential requests) or "1.1" (one), from the offer |
| Where the credential is | in the head of every request, written by N | written by N once per connection; on HTTP/2 later requests name it by index |
| Two-party key schedule | garbled in full: 31 compressions, 656,179 AND gates, 22.1 MB down | pad states and outer compressions: 22 compressions, 479,022 AND gates, 16.4 MB down |
| Proof about C's records | $\pi_{C2S}$: the tail of one record | $\pi_{WIN}$: any windows of any records |
| Usage anchoring | the end of the transcript | the request's interval of server records |
| Held at the setup | a full-context reservation | a connection deposit |
| Reservation | per session, at the setup | per request |
| Session fee | per request | once per connection |
| Pre-warming | sessions bound to model, cap and body length | an armed connection bound to the credential; a kept connection on HTTP/2 |
| Selective-failure bound | $\log_2 7 \approx 2.8$ bits per request | 3 bits per connection |
| Assumption on the hash | HMAC-SHA-256 is a PRF | two compression-function assumptions (section 4.4) |

**Why HTTP/2.** The credential must be written under a key only N holds, and that key comes from a single-use garbled key schedule, so N can write once per handshake. HTTP/1.1 repeats the credential in every request head; keep-alive would not help, because the second head would again need a key C does not have. We considered giving N a fresh write key per request (a KeyUpdate chain computed jointly, or a per-request 2PC of one derivation step), which keeps a

garbled circuit on every request, and session resumption, which needs a new joint key schedule per connection anyway. HTTP/2's header compression offers what is needed without new cryptography: a header can be stored once in the receiver's table and named afterwards by an index that reveals nothing about its value. The provider is unmodified; it sees a client that sends a probe request, a KeyUpdate, and then ordinary requests with a compressed header.

**What it costs.** C's records now carry instructions to an HTTP/2 stack, so the proof obligation grew from "the last record is the suffix" to the frame rules of section 4.6, and the usage disclosure needs the interval rule of section 4.7. Requests on one connection are linkable for the provider (residual risk 16). The provider must satisfy more (section 3.1). None of this has had external review.

**Versions.** v3 replaces v2; there is no fallback between them. A v3 node offers only version 3 and answers a v2 client's socket with an explicit "upgrade required" error naming the minimum version. A v3 client that reads an offer of a lower version does not use that node for confidential requests and reports the node as outdated. The node's listing carries the version, so clients can filter before fetching offers. Within v3, the fallback is between framings on the same node (section 4.2).

## 7.2 v1: the single-party variant

The protocol had a simpler variant, which we call v1, without any 2PC. It was implemented and measured and is retired: no current code speaks it. It remains on the `v1-poc` branch of routstr-confidential. We describe it because it shows what the 2PC buys.

**How it works.** C runs the TLS handshake alone, with an ordinary X25519 key share, and discloses only $c_{hs}$ and $s_{hs}$ to N, which verifies the handshake as in v2. C then sends a KeyUpdate in epoch 0, so that epoch 1 starts. For epoch-1 sequence 0, C gives N **one-shot record material**: the CTR keystream truncated to the head length, the GHASH key and the tag mask. $\pi_{C1}$ proves to N, with HS as the witness, that this material comes from the session's key schedule (HS through MS, $c_{ap,0}$, $c_{ap,1}$, $K_1$, $IV_1$), so that C cannot hand out a keystream that rewrites the head around N's key. N forms its record by XOR, proves its content with $\pi_N$ (today's $\pi_{NK}$ with a public keystream), and writes it; C then sends a second KeyUpdate and writes the body in epoch 2. $\pi_{C2}$ and $\pi_{C3}$ prove the suffix and the usage event like their v2 counterparts, but take HS as the witness and anchor on the public handshake values, so each recomputes the key schedule (28 to 36 compressions).

**What it gives up.** C holds the whole epoch-1 key $K_1$: it needs it to seal its own KeyUpdate at sequence 1. The API key is therefore exactly $C_1 \oplus S$, and P1 holds only as long as C never sees $C_1$. The v1 implementation never sent it to C, but anyone on N's egress path who colludes with C (a shared cloud network, a TLS-inspecting middlebox, the hosting provider, a CDN edge) can recover the key, and one leak exposes it until it is rotated. Short-lived keys with spend caps limit the loss without removing the assumption. In v1 P1 is an operational assumption; since v2 it is a cryptographic property. P2 to P6 are the same in both versions, and v1's residual risk 1 is narrower: N can write exactly one record of fixed length.

| | v1 | v3 |
|---|---|---|
| Client can recover the API key | yes, if it sees N's egress ciphertext | no |
| New cryptography in the trust base | ZK proofs | ZK proofs and a 2PC |
| Key exchange | X25519 | P-256 |
| Data per request | ≈5.9 MB | 16.4 MB of 2PC plus proofs per connection; 0.5-1.0 MB per later request on HTTP/2 |
| Suffix proof on the request path | 0.47 s local | 39-66 ms over the proof channel |

v1's bandwidth advantage holds only for one-request connections. On a kept HTTP/2 connection v3 moves less data per request than v1 did from about the fifth request on (with the connection open as measured in section 6.3).

# 8. Deployment pattern

Confidential mode plugs into an existing routing deployment as an alternative *transport*; everything else stays the deployment's own code. Three legs are involved, and the framing of section 4.2 is a property of the inner TLS connection only; the application's HTTP does not change:

```
app --HTTP/1.1, OpenAI API--> client daemon --WebSocket--> node --> relay --TCP--> provider
                                  |_________ TLS 1.3, framing "2" or "1.1" ___________|
```

1. **Offer.** The node publishes a signed offer at a public endpoint: protocol version (3), the framings in order of preference and the HTTP/2 parameters (probe path, requests per connection, lifetime), the key-schedule protocol, upstream hostname, key length and alphabet, the HTTP/1.1 head template, the description of the pinned suffix, a signed rate snapshot, the connection deposit, the idle timeout, the fees, and the attestation key. A node that serves images adds the `images` capability and its signed per-image variants (section 4.10). The framings and HTTP/2 parameters come from the relay's offer file, where the operator states what the upstream supports.
2. **Model listing.** The model listing gains one additive field per model that references the offer; old clients ignore it.
3. **Opt-in.** A client opts in per request, for `POST /v1/chat/completions` and, on nodes with the `images` capability, the image routes. It routes only to nodes that advertise the field for the model, and requires that the offer signature verifies with a pinned node key (not just the key the offer names): one the user configured, or else the key that announced the node's URL on Nostr, for an https or onion URL only, where fetching the offer authenticates the server. (Plain http is accepted only for a loopback node, for development; the session carries the node credential.) It also requires that the upstream hostname is on its trusted hostname list (an empty list refuses every host), and that its prover is available. It rebuilds every public input of $\pi_{NK}$ from the signed offer. It also checks that the offer's version is 3, that it implements the named key-schedule protocol, that the HTTP/2 parameters are in range, and that the deposit is at most its configured maximum.
4. **Billing** is the node's ordinary billing (section 4.8). The client's credential, reservations, settlement, balance and refunds are the existing ones. In the reference integration the node adds a WebSocket route and an offer; the client SDK adds one transport branch beside its other transports, with the connection pool behind it, so credential acquisition, top-ups, failover and balance accounting are unchanged, except that a node error after a body is released is not retried on another node (section 4.9); the daemon adds a configuration block and passes the opt-in through.
5. **Response.** The client streams the decrypted body to the application as records arrive, or folds the stream into one completion object if the caller did not ask for streaming (the pinned suffix always requests streaming, because the usage event only exists there). Checks that complete after the response (the disclosure proof, the receipt, the usage, the cost, the served model) are not dropped: a failure fails a non-streaming call and ends a stream with an error.

For existing deployments the change is additive: the ordinary proxy path and old clients are untouched. Requests that opted in fail closed and are never sent on the plain path, since a silent downgrade would break the privacy promise.

**Pool and reuse scope.** The daemon serves several local applications. By default it gives each local client (each local API key) its own reuse scope, so two applications never share an upstream connection and the provider cannot link them through one (residual risk 16); the setting can also share connections across all local clients or disable reuse.

## 8.1 Denial-of-service protection and fees

Opening a confidential connection is expensive for the node before the provider sees a byte: N garbles the key-schedule circuit and sends about 16 MB, spends about 0.4 s of CPU and holds about 240 MiB more memory while the connection's 2PC runs. A request on an open HTTP/2 connection is cheap: two short proofs. The limits follow that split.

1. **Nothing reaches the provider before a ClientHello.** The relay dials U only when C's ClientHello arrives, after setup and the 2PC preprocessing, so armed and abandoned connections never open provider connections.
2. **Validation and deposit before work.** N validates the setup and the credential and reserves the connection deposit before it contacts the relay. A client without funds gets no garbled circuit.
3. **A wasted connection is charged.** The deposit is released by the first settled request and charged when a connection ends without one (section 4.8). In v2 the same role was played by the abandon fee on an unused session;

the deposit separates it from the request, so a paying user never pays it and an armed connection no longer holds a full-context reservation.

4. **Reservation per request.** Each request is validated and reserved before N accepts its records, at the worst case for its model (section 4.6). A refused request costs neither side a handshake.
5. **Caps.** N serves a bounded number of concurrent connections (32 by default), each for at most `max_lifetime_s` and `max_requests`, and closes idle ones after `idle_timeout_s`. A connection counts against its address's limit of unused connections (twelve by default) until its first body is released. Connections whose setup is not reserved yet are a separate pool (at most 4 per address and 1024 in all) in which the oldest is evicted first, so idle sockets cannot keep a paying setup out. The address is the TCP peer, or, only for peers configured as trusted reverse proxies, the rightmost `X-Forwarded-For` entry. Clients behind one address, such as all Tor users of an onion service, share the limit.
6. **Fees in the signed offer (section 4.8).** A session fee $f_s$ once per connection, an abandon fee $f_a \geq f_s$ on a reserved request that is dropped before its body is released, and a rate surcharge, all 0 by default. C must echo the fees it accepts, so a client that does not know about them is refused, not charged.
7. **Client-side restraint.** The client opens connections only for requests, keeps at most one armed connection per pool key and only while there is traffic and every open connection is busy, accepts a deposit only up to a configured maximum, closes idle connections before the node does, and logs every deposit it loses.

**How the deployed node is set up.** The deployed case-study node runs v3 since 2026-11-12 (framings `"2"` and `"1.1"`). It charges no session fee and no surcharge, and an abandon fee of 2 sats, which under v3 is also its connection deposit (2000 msat). A typical confidential chat on its cheapest models costs 0.1 to 1 sat of inference: completed requests of up to 40 output tokens on two DeepSeek models were charged 11 to 27 msat on 2026-10-19 (signed v2 receipts in `docs/evidence/deployed-receipts/`). So a paying user pays exactly the inference, and only waste is priced. Each chat request reserves the model's worst-case prompt for its duration (on 2026-10-19, 119 to 160 sats for models with a context of about 1M tokens) and releases what it does not use, so a credential needs that much balance to send a request even though it is not spent. Under v3 that reservation exists only while a request is in flight. On the deployed v3 node (2026-11-12, session fee 0, deposit 2000 msat) the same model's requests of 15 prompt and 30 output tokens were charged 10 msat each against a reservation of 133 410 msat, and the deposit came back with the connection's first receipt.

**What remains.** The deposit and the limits bound the cost per connection and per address, not the total: many addresses can still open connections, each risking a deposit, which is a priced service and not an attack. An operator who wants a different trade-off raises the deposit, lowers the per-address limit, or charges a session fee to cover the node's bandwidth.

# 9. Related work and alternatives

**MPC-TLS (TLSNotary).** The prover and a notary jointly act as the TLS client, and all record encryption and decryption runs inside the MPC. With the roles arranged so that the node is the notary, it gives prompt confidentiality and lets the API key be the node's private input, and it works with TLS 1.2. The cost scales with the payload and there is no streaming: we measured about 5.8 s and 41.5 MB for a 2 KB request, 34 s and 377 MB for 32 KB, and 129 s and 1.44 GB for 128 KB (100 Mbit/s, 50 ms RTT, emulated; one sample per size, against TLSNotary's local TLS 1.2 test server rather than an LLM provider; raw log in `docs/evidence/mpc-tls-2026-10-09/`). Most of that time is preprocessing, which can run ahead; the part after the connection opens took 1.5 s to 15 s. Our 2PC moves under half the data of the 2 KB case, once per connection, and it covers only the key schedule, so its cost is constant, can be precomputed, and the payload then streams at native speed. TLSNotary's key-exchange conversion is the basis of our ECtF.

**DEAP and Garble-then-Prove.** TLSNotary's dual execution with asymmetric privacy (DEAP) and Garble-then-Prove (Xie et al., USENIX Security 2024) obtain malicious security cheaply by having one party open its inputs, or its key shares, after the session, so that the other can check the garbling and later prove statements about the transcript. That fits a notary whose shares are worthless after the session. It does not fit our setting: N's share would reveal the master secret, hence $K_0$, hence the API key, which is a long-term secret. We need a 2PC in which nothing is ever opened. Our first version used full dual execution with an equality check on output labels (Huang, Katz and Evans 2012; section 6.6); the current one uses garbler-only authenticated garbling.

**Authenticated garbling** (Wang, Ranellucci and Katz 2017; Katz, Ranellucci, Rosulek and Wang 2018; Dittmer, Ishai, Lu and Ostrovsky 2022; Cui, Wang, Yang and Yu 2023) makes one garbled circuit secure against a malicious garbler. The wire masks are secret-shared between the parties and the AND gates are backed by authenticated triples, so nothing leaks. KRRW-style triple generation costs more bandwidth, in both directions, than garbling twice, and the compressed preprocessing of DILO and CWYY needs primitives (block correlated OT or subfield VOLE with many keys) that `mpz2` does not provide. We keep the masks with the garbler and authenticate them to the evaluator. That needs only correlated OT and a QuickSilver-style product check, and its batched per-wire check follows CWYY's, but it accepts a selective-failure leakage of at most 3 bits per connection (given a semi-honest share conversion that leaks nothing beyond its outcome). Its soundness argument is our own (section 5). Three-halves garbling (Rosulek and Roy 2021) cuts N's tables by about a quarter against half-gates.

**DiStefano** (Celi et al., 2023) runs the TLS 1.3 handshake as a 2PC with fully authenticated garbling, which is maliciously secure without leakage but needs heavier preprocessing. It could replace our 2PC where the remaining bits of leakage matter more than bandwidth.

**DECO** (Zhang et al., ACM CCS 2020) introduced the three-party handshake, in which prover and verifier secret-share the session keys so that the verifier is convinced of data the prover received. Its goal is to convince the verifier of provenance; ours is the reverse split, in which each party receives a different epoch's key in full and the payload never enters the MPC.

**Header compression as an attack surface.** CRIME (Rizzo and Duong 2012) and BREACH recover secrets from the length of data compressed together with attacker-chosen text. HPACK (RFC 7541) was designed to resist this by matching whole header fields only. We rely on the same property from the other side: the party that holds a reference to a secret header (C) is the one who must not learn it, and whole-field references give it nothing to measure (section 5).

**Proxy zkTLS attestors** (the "verified mode" deployed on Routstr nodes; Reclaim-style attestors). The node is the TLS client and an attestor checks its transcript. This gives request and response integrity and portable proofs, but the node and the attestor see the plaintext; we measured +10.8 s to first token on a live node (another node and upstream than our case study; author measurement, raw log not archived).

**TEE (enclave) designs.** The node runs in a hardware enclave that holds the API key and processes the prompt in the clear, and users verify attestations. This is fast and streams, and it works with any provider. It rests on the hardware vendor and on the measured code, and the guarantee is only as good as the user's review of every software version the enclave runs (section 1).

The two approaches compose. Our protocol removes the need to trust the reseller in the middle, while C remains the TLS endpoint of the provider. If the provider itself serves inference from an enclave and binds its TLS key or an encryption key to an attestation (attested or end-to-end-encrypted inference offered by some providers), C can verify that attestation end to end through N, which only relays sealed records. The user then gets math-based guarantees about the node and the provider's attestation about the provider, without having to review the node's software.

| | This work (v3) | v1 variant | MPC-TLS | Proxy zkTLS | TEE |
|---|---|---|---|---|---|
| Prompt hidden from node | yes | yes | yes | no | yes |
| API key hidden from client | yes | if N's egress is private | yes | yes | yes |
| Trusted hardware or attested software | no | no | no | no | yes |
| Streaming, native-speed payload | yes | yes | no | yes | yes |
| Added latency on the request path (local) | ≈60 ms on a kept HTTP/2 connection | 0.45 s proof, pre-warmed | 1.5-15 s (100 Mbit/s) | ≈10 s | ≈0 |

| | This work (v3) | v1 variant | MPC-TLS | Proxy zkTLS | TEE |
|---|---|---|---|---|---|
| Data per request | 0.5-1 MB on a kept connection; ≈24 MB to open one (measured with the v2-size 2PC) | ≈5.9 MB | 41 MB-1.4 GB | transcript size | ≈0 |
| Third-party-verifiable transcript | no | no | yes | yes | attestation only |
| Provider requirement | TLS 1.3 profile with P-256; the HTTP/2 items for kept connections | TLS 1.3 profile | TLS 1.2+ | TLS 1.2/1.3 | any |

The approaches compose: proxy zkTLS provides integrity for unmodified providers, MPC-TLS covers providers that fail the profile, and our design is the fast lane where a provider's TLS and JSON semantics cooperate.

# 10. Conclusion

We presented a protocol that gives LLM routing users prompt confidentiality, request and response integrity, and upstream-anchored accounting against a node that relays ciphertext, and gives the node a guarantee that its API key stays hidden from the client, with no trusted hardware and no MPC over the payload. Its trust rests on the provider's TLS certificate and on cryptography, not on attested software. The mechanisms are a two-party TLS key schedule that gives the node the first application epoch and the client every other one, a credential that the node writes once per connection and that later HTTP/2 requests name by a header-table index, a window proof that keeps the client's records to the frames and the public suffix they must be, and a proven usage disclosure for settlement.

The two-party handshake costs 16.4 MB to the client and 1.1 MB back, and about 7 s on a home link with a 2 Mbit/s uplink. In the previous version every request paid it. Now a connection pays it once: in our runs against a live provider, a later request on an open HTTP/2 connection moved 0.5 to 1.0 MB and was released to the provider about 60 ms after it was sent. Where a provider or a client cannot use HTTP/2, the same lifecycle carries one HTTP/1.1 request per connection at the cost of the handshake.

The price is a larger argument. The key schedule exposes the inner HMAC states to both parties and rests on two compression-function assumptions; a cheating node can learn at most 3 bits per connection about values that are never reused; and the HTTP/2 frame rules, the window proof and the read-back argument for the header table are new. None of these has had external review, one provider has been tested, and that is the work that remains.

---

# Appendix A. Case study: the provider profile in practice

**A.1 Verified behaviours** (live gate `crypto/tools/provider-profile-check.py`, checks A1-A12, 12/12 on 2026-10-19 with a completion margin of 16; rate limits, 5xx and timeouts count as inconclusive, never as a pass, and each rejection check asserts the exact status and error text against a valid control). The token-cap overshoot, the system-prompt pin and the fake usage events are manual tests outside the gate. The image route's gate (`--images`, I1 to I9, 10 PASS on 2026-10-19) and a direct image session log are there too. The outputs of that day's runs are in `docs/evidence/`.

- **Transport:** a client offering only TLS 1.3 with `TLS_AES_128_GCM_SHA256` is served; a client KeyUpdate sent in the middle of the request is accepted and the server answers completely and closes (`Connection: close`,

HTTP/1.1). The gate offers X25519; the v2 live sessions establish that a single secp256r1 key share is accepted without a HelloRetryRequest.

- **Waits for the full body:** nothing for 8 s with the body incomplete, then a response within 0.7 s of the last bytes.
- **Last-wins:** a bad model then a good one gives 200; a good one then a bad one gives 404; a `max_tokens` of 200 followed by 8 behaves like 8.
- **Replace, not merge:** a later `venice_parameters` object replaces an earlier one; an invalid `character_slug` that is overridden is ignored.
- **Strict parsing (sampled):** an unknown top-level field and malformed JSON are rejected with 400, including a suffix landing in an open object or string. The 400 body for an unknown field quotes the field's name, which is why an error disclosure covers only the response head (section 4.8).
- **Token cap:** `max_completion_tokens` caps output including reasoning, with a small overshoot on tiny caps (3 billed up to 6, 40 billed 39). The reservation's completion margin (16 tokens by default, section 4.6) covers this.
- **Input bound:** with the pinned `include_venice_system_prompt: false` a 224-byte request billed 5 prompt tokens. Without the pin the provider injects about 1.7k tokens of system prompt.
- **Unpinned price fields (sampled):** `service_tier`, `web_search_options` and `prediction` are rejected.
- **Image input:** vision models accept image parts in the private body (a remote `image_url` or an inline data URI) and report the image as prompt tokens: a 256x256 PNG adds about 70 to 110 on mistral-small and qwen3-vl and about 1,090 on gemini-3-flash-preview. On 2026-10-26 the account balance fell by exactly the reported tokens at the listed rates for 15 image requests and 15 text requests (qwen3-vl-235b-a22b), so the settlement at the frozen input rate covers images; the full-context reservation bounds them before the answer. Gate check A13 (added 2026-10-27; 13/13 on 2026-10-28) asserts that an image raises `prompt_tokens` and that the usage event carries nothing the node does not price. Text-only models reject image parts with a 400; the provider refuses images under 64x64 on every model, so an earlier data-URI check that sent a 1x1 PNG proved nothing (`crypto/tools/regression-modality.py` now sends 64x64).
- **HTTP/2 (2026-11-11, manual checks before the `--h2` gate existed):** the API host selects ALPN `h2`; a TLS KeyUpdate directly after the first flight is accepted; `HEADER_TABLE_SIZE = 0` is honoured for response headers; a 200 kB body passes flow control; a request with an index-only `authorization` is authenticated, also after 150 s of idle; no part of the credential appeared in eight response paths.
- **Usage event:** the terminal event carries usage, cost and model with empty `choices`, followed by `[DONE]`.
- **Event framing:** the dechunked stream uses LF line endings only, and every event is one `data:` line of JSON followed by a blank line, ending in `data: [DONE]`: the framing the usage proof parses.
- **Fake usage events:** a prompt instructing the model to emit them produced none. Every imitation was escaped inside a JSON string, and the raw stream contained no CR/LF inside events.

**A.2 Profile application.** The pinned suffix for this provider, as its offer describes it: `,"model":M,"max_tokens":T,"max_completion_` with `max_completion_tokens` pinned because it overrides `max_tokens` (a bypass observed in testing) and the provider-specific parameters pinned off to keep web search and system-prompt injection off the bill. Another provider's offer leaves out `venice_parameters` and pins its own extras off instead.

**A.3 Other providers.** The generic pieces (epochs, 2PC, proofs) port unchanged. The open questions per provider are profile items 4 to 6, 8, 9, 11 and 12, plus whether P-256 and a KeyUpdate are accepted and which framing the front end serves: HTTP/1.1, or HTTP/2 with items H1 to H8 (a front end that serves only HTTP/2 is covered by framing "2" when it keeps the dynamic table). Where a provider fails, the fallbacks are MPC-TLS for confidentiality or the existing integrity-only path. Per-provider assessments are in the project's `docs/ADVERSARIAL-QA.md`.

# References


- S. Celi, A. Davidson, H. Haddadi, G. Pestana, J. Rowell. *DiStefano: Decentralized infrastructure for sharing trusted encrypted facts and nothing more.* NDSS 2025 (ePrint 2023/1063).
- H. Cui, X. Wang, K. Yang, Y. Yu. *Actively secure half-gates with minimum overhead under duplex networks.* EUROCRYPT 2023.
- S. Dittmer, Y. Ishai, S. Lu, R. Ostrovsky. *Authenticated garbling from simple correlations.* CRYPTO 2022.
- Y. Huang, J. Katz, D. Evans. *Quid-Pro-Quo-tocols: Strengthening semi-honest protocols with dual execution.*

IEEE S&P 2012.
- J. Katz, S. Ranellucci, M. Rosulek, X. Wang. *Optimizing authenticated garbling for faster secure two-party computation.* CRYPTO 2018.
- M. Bellare. *New proofs for NMAC and HMAC: Security without collision-resistance.* CRYPTO 2006.
- R. Peon, H. Ruellan. *HPACK: Header compression for HTTP/2.* RFC 7541, 2015.
- J. Rizzo, T. Duong. *The CRIME attack.* ekoparty 2012.
- M. Thomson, C. Benfield. *HTTP/2.* RFC 9113, 2022.
- M. Keller, E. Orsini, P. Scholl. *Actively secure OT extension with optimal overhead* (KOS). CRYPTO 2015.
- P. Mohassel, M. Franklin. *Efficiency tradeoffs for malicious two-party computation.* PKC 2006.
- E. Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3.* RFC 8446, 2018; test vectors RFC 8448.
- M. Rosulek, L. Roy. *Three halves make a whole? Beating the half-gates lower bound for garbled circuits.* CRYPTO 2021.
- TLSNotary project. *mpz* and *mpz2* (garbling, oblivious transfer, VOLE-based ZK) and the MPC-TLS protocol documentation.
- X. Wang, S. Ranellucci, J. Katz. *Authenticated garbling and efficient maliciously secure two-party computation.* ACM CCS 2017.
- R. Weiss, D. Ayzenshteyn, G. Amit, Y. Mirsky. *What was your prompt? A remote keylogging attack on AI assistants.* USENIX Security 2024.
- X. Xie, K. Yang, X. Wang, Y. Yu. *Lightweight authentication of web data via garble-then-prove.* USENIX Security 2024.
- K. Yang, C. Weng, X. Lan, J. Zhang, X. Wang. *Ferret: Fast extension for correlated OT with small communication.* ACM CCS 2020.
- K. Yang, P. Sarkar, C. Weng, X. Wang. *QuickSilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field.* ACM CCS 2021.
- S. Zahur, M. Rosulek, D. Evans. *Two halves make a whole* (half-gates). EUROCRYPT 2015.
- F. Zhang, D. Maram, H. Malvai, S. Goldfeder, A. Juels. *DECO: Liberating web data using decentralized oracles for TLS.* ACM CCS 2020.